# FitFlow: An AI-Powered Fitness Coaching Platform with Computer Vision Form Analysis

**By**

Emam Ul Haque Emon

ID:222-15-6290

**FINAL YEAR DESIGN PROJECT REPORT**

This Report Presented in Partial Fulfillment of the Requirements for the **Degree of Bachelor of Science in Computer Science and Engineering**

**Supervised by**

**Professor Dr. Sheak Rashed Haider Noori**

**Professor & Head**

Department of Computer Science and Engineering Daffodil International University

**Co-Supervised by**

**Mr. Md. Hasanuzzaman Dipu**

**Assistant Professor**

Department of Computer Science and Engineering Daffodil International University


**DAFFODIL INTERNATIONAL UNIVERSITY**

**Dhaka, Bangladesh**

**April 11, 2026**

# APPROVAL

This Project titled "**An AI-Driven Fitness-as-a-Service Ecosystem for Real-Time Workout Intelligence, Personalized Coaching Strategies, and Scalable Health Performance Analytics**," submitted by **Emam Ul Haque Emon** to the Department of Computer Science and Engineering, Daffodil International University, has been accepted as satisfactory for the partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Engineering and approved as to its style and contents. The presentation has been held on **11-05-2026.**

**BOARD OF EXAMINERS**

---

**Name**

**Board Chairman**

Designation, Department of CSE, FSIT
Daffodil International University

---

**Name**

**Internal Examiner 1**

Designation, Department of CSE, FSIT
Daffodil International University

---

**Name**

**Internal Examiner 2**

Designation, Department of CSE, FSIT
Daffodil International University

---

**Name**

**External Examiner**

Designation, Department of CSE, FSIT Daffodil International University

# DECLARATION

We hereby declare that this project has been done by us under the supervision of **Professor Dr. Sheak Rashed Haider Noori, Professor & Head** , Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere for the award of any degree or diploma.

**Supervised by:**

**Professor Dr. Sheak Rashed Haider Noori**
**Professor & Head**
Department of Computer Science and
Engineering Daffodil International University

**Co-Supervised by:**

**Mr. Md. Hasanuzzaman Dipu**
**Assistant Professor**
Designation
Department of Computer Science and
Engineering Daffodil International University

**Submitted by:**

**Emam Ul Haque Emon**
Student ID: 222-15-6290
Department of Computer Science and
Engineering Daffodil International University

# ACKNOWLEDGEMENTS

This work would not have been possible without the support and contributions of many individuals over the past two semesters. We are deeply grateful to everyone who has assisted us in one way or another.

First, we express our heartfelt thanks and gratefulness to the almighty for His divine blessing making it possible for us to complete the **Final Year Design Project(FYDP)** successfully.

We are grateful and wish our profound indebtedness to **Professor Dr. Sheak Rashed Haider Noori, Professor & Head** Department of Computer Science and Engineering, Daffodil International University, Dhaka, Bangladesh. Deep knowledge and keen interest of our supervisor in the field of **Web and App Development** to carry out this project. His endless pa- tience, scholarly guidance, continual encouragement, constant and energetic supervision, constructive criticism, valuable advice, reading many inferior drafts, and correcting them at all stages have made it possible to complete this project.

We would like to express our heartfelt gratitude to the Head of the Department of Computer Science and Engineering, for his kind help in finishing our project and also to other faculty members and the staff of the Department of Computer Science and Engineering, Daffodil International University.

We would like to thank our entire course-mates at Daffodil International University, who took part in this discussion while completing the coursework.

Finally, we must acknowledge with due respect the constant support and patience of our parents.

# ABSTRACT

This report presents FitFlow, a web-based fitness coaching platform that combines real-time computer vision form analysis, an AI conversational coach, and a multi-role social graph linking trainees, certified coaches, and platform administrators. The motivation comes from a gap in existing fitness applications: free apps count workouts but cannot tell a user whether their squat is deep enough, while in-person coaching is expensive and geographically constrained. The platform addresses this by running a MediaPipe Pose [1], [3], [26] pipeline entirely in the browser, scoring squat, push-up, and plank form against angle-based heuristics and a smoothed exponential decay model, then surfacing the resulting metrics in a personal dashboard with thirty-day trends. A separate role layer turns the same app into a coaching tool: trainees can search a verified coach directory, request a link, and exchange messages once accepted; coaches see a productivity-focused dashboard with today's clients, recent sessions, and clients who have not trained in seven days. Administrators have a parallel surface for moderating users and reviewing coach applications. The system is implemented in Next.js 14 with the App Router [28], MongoDB [27] through Mongoose, JWT-based authentication [11], role-aware middleware, and a Stripe [29] billing integration that gates the AI conversational endpoint [30] behind a Pro tier. Validation, rate limiting, structured logging, and an OpenAPI 3.1 specification [16] round out the production posture. Testing focused on functional correctness of the role transitions, latency of the pose-detection pipeline on a mid-range laptop, and accuracy of the rep counter against hand-counted ground truth across three exercises. The platform is positioned as a foundation that future work can extend with calibration-based form scoring, client-side mobile capture, and richer coach analytics.

# Table of Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

The title of the project introduced in this chapter is "FitFlow: An AI-Powered Fitness Coaching Platform with Computer Vision Form Analysis." This chapter sets the scene: what the project is, why it exists, what it is trying to do, and how the rest of the report is organized. It does not assume that the reader has already used a fitness app or thought about computer vision. It builds the case from the ground up.

## 1.1 Introduction

FitFlow is a web-based fitness coaching site. It is powered by Next.js 14 [28] on the front and the API layer, MongoDB [27] via Mongoose on the persistence layer and a MediaPipe Pose [1], [26] detection pipeline that fully runs in the browser on the client machine. There are three types of users of the product, the trainees who train, the coaches who review and guide trainees and the administrators who keep the platform healthy and each role is dropped on a different home page after logging in. The trainees are shown a dashboard, where their thirty days of progress, streak and live workout shortcut are shown. Today's clients, recent sessions of all the connected clients and a list of the clients who have not been training within seven days are all seen by the coaches. Administrators see platform stats and queues for moderation.

The live workout module is the technical centre of the platform. The trainee opens the camera (W3C MediaCapture/Streams [31]) then picks an exercise (squat, push-up, or plank) and the browser starts to execute MediaPipe Pose [1], [26] with a frequency of about 30 frames per second. The landmark stream is fed into a small per-exercise state machine that transforms the stream into reps, hold time, calorie estimates, and a smoothed form score. The video does not leave the device, but the resultant numbers are saved to the server when the session is over. It is meant to provide something that is more or less like a coach watching you without all the bandwidth or privacy concerns that streaming video to a remote model can bring.

At the very top of this technical core is a social layer. Those trainees who would like to be coached can browse a list of approved coaches, see the biography of a coach, send him or her a connection request and interact through messages. Coaches are eligible to have clients and submit an application at sign-up and then can be discovered once such an application is approved by an administrator. Brief check-in, technique reminders, and progress notes can be part of the same conversation. The platform intentionally keeps the coach notes secret to the

trainee and show the trainee messages to the coach, as is the case with in-person coaching.

## 1.2 Motivation

This project came about as a result of two observations. The first are free fitness apps which inform the user on what to do but do not tell the user how they did it. They record reps when you press a button. They possess a collection of YouTube-like demonstration videos. They are non-observant. Any person who has attempted to learn a squat using a video knows the difference: the video demonstrates a perfect squat, and the user performs something that either is or is not a squat with no feedback in either way. The outcome is the familiar figure of harm and disappointment which fitness applications silently generate.

Second, real coaching not only gets the feedback problem solved but also gets another problem solved. A certified coach in Dhaka not only charges per session but also demands a geographic proximity and only observes the client during the scheduled hours. The fact that most reps occur between training sessions makes the coach have no insight into which work is the most important. A platform capable of performing some form of inter-session partially form analysis, surfacing that information to the coach and letting coach and trainee message about what came up, would make the human coach much more useful per hour.

FitFlow is at the cross-section. The computer vision on the browser side gives feedbacks in all the workouts. The coach layer takes as raw material that feedback and uses it to asynchronously coach. The architectures view the AI as an assistant to the coach, rather than a substitute to the coach.

## 1.3 Objectives

- The purposes of FitFlow are purposely limited in such a way that each of them can be justified by the solid facts presented in this report:
- Include correct-enough rep counting and form scoring of three exercises (squat, push-up, plank) that run completely in the browser, with no video processing on the server.
- Keep a thirty day history of number of sessions per trainee and surface trends -current streak, average form score, total reps, session by exercise on the trainee dashboard.
- Use role-aware routing, role-based API guards and a JWT-based session to carry the role.
- Provide certified coaches with a working surface directory listing, application flow with a approval of the administration, client roster, per-client trends, private coach notes, and direct messaging.

- Provide administrators with a moderation surface user list with role and suspension control, coach application queue, audit log of role changes - enough to access the platform without database access.
- gate the OpenAI-powered chatting assistant behind a premium Pro tier with a Stripe checkout flow and webhook-driven entitlement, and show an earnest approach to the sustainability of its platform.

## 1.4 Methodology

1. The project was done using an iterative-waterfall hybrid. The chapter structure of this report itself, i.e. requirements were gathered, the system was designed, modules were implemented, and the result was tested, is the waterfall part. The iterative component is the one that occurred on a weekly basis. Each module, such as auth, live workout, coach surface, messaging, and a small acceptance test, was developed as a vertical slice end-to-end, including UI, API, persistence, and a small acceptance test, before transitioning to the next slice. The advantage was that at each checkpoint the application could be shown and not some half-finished set of layers.
2. The key events, in the sequence that they occurred, were:
3. Identification of problems and competitor analysis. The available fitness platforms were listed and their missing areas identified. Their narrowing was not without purpose: not build a fitness app but build the components of a fitness app that a free app does not do well.
4. Requirement specification. Functional and non-functional requirements were documented prior to code, and then again twice with the build. Both versions are reflected in chapter 3.
5. System design. The source of truth was sketched manually on a piece of paper and then converted into TypeScript interfaces and Mongoose schemas.
6. Implementation. By order, vertical slices include: authentication, trainee dashboard, live workout, progress dashboard, coach surface, admin surface, messaging, billing.
7. Testing. Type checking at the unit level using the TypeScript compiler, integration at the unit level by testing each role flow manually, and ad-hoc accuracy testing of the rep counter against hand-counted ground truth of three exercises.

8. Polishing. Once all was working, the visual layer was built again, replacing the default shadcn look with one using hero gradient cards each on a major page.

## 1.5 Project Outcome

- The final product is a functional web app. A trainee is able to register, be guided by a workout in front of his or her laptop camera, and instantaneously see his or her session reflected in his or her progress dashboard. A coach can sign-up, be accepted by an administrator, update their public bio, accept an incoming request, see their client activity of the day, drill into the form-score trend of a particular client, and send a message to the client. An administrator is able to approve, suspend, and promote users and live statistics of the platform.
- In particular the build yields:
- An interactive, browser-based pose-detection pipeline capable of scoring squats, push-ups and planks at interactive frame rates on a mid-range laptop.
- The resultant session log is converted into a thirty-day progress narrative, including streak, average form, and per-exercise breakdown.
- The surface of a coach dashboard displays the clients of the day, the recent sessions of the roster, the clients who need to be addressed and a profile-health indicator.
- A dashboard of an administrator with platform statistics, user management, and the coach-application review queue with approve and reject (with reason) options.
- A direct-messaging surface that is accessible to the coach and the trainee once they are connected, with thread list, composer, day-grouped bubbles and an unread badge in the navigation bar.
- A billing workflow using Stripe checkout, a customer portal, and entitlement that gates the AI conversational endpoint using the Pro tier.

## 1.6 Organization of the Report

The report is divided into six chapters which conform to the recommended FYDP format.

Chapter 1: Introduction. This chapter. Outlines the project, motivation, objectives, methodology, outcomes and structure of the document.

Chapter 2: Background. Scans the scene of the fitness-app and computer-vision-coaching landscape, summarises a set of related works, and runs a gap analysis, which justifies why this project was worth building.

Chapter 3: Research Methodology. The data model, data flow diagrams at three levels of detail, data model, the use-case diagram, the selected development model, the project plan, and the task allocation timeline.

Chapter 4: Implementation and Results. The tooling, setting up the environment, the testing plan, the numbers related to the performance and a comparison to the alternatives that were surveyed in chapter 2.

Chapter 5: Engineering Standards and Design Challenges. Software, hardware and communications standards compliance; societal, environmental, sustainability impact; budget and revenue analysis; mapping to complex engineering problems attributes.

Chapter 6: Conclusion. What the project accomplished, what it failed to accomplish and where the follow-up version ought to be taken.

# Chapter 2

# Background

The chapter sets a research and product background to the project. It starts with a concise investigation concerning the reason why digital fitness platforms have become a serious category, surveys ten relevant works of the literature, compares five real-world products and ends with a gap analysis that justifies the design choice in chapter 3.

## 2.1 Introduction

The fitness software market has expanded due to three factors that came together around approximately the last 10 years: smartphones have become powerful enough to be able to run real-time pose models on-device, the pandemic-era restrictions normalized at-home training, and consumer-grade wearables made longitudinal-personal data the new norm. The outcome of these three forces was a category that is now worth several billion dollars a year and is projected to continue to increase.

Within that category, there are two groups of products that prevail. The first one is the workout-library cluster: the apps such as Nike Training Club, FitOn, and Peloton App provide guided video workouts and tracking-by-tap. The second one is the wearable-data cluster: Apple Fitness+, Garmin Connect, and Whoop transform heart-rate variability and movement into long-term insights. Both groups omit the question that is of the greatest concern to a novice: am I doing this exercise right? In neither category does the app actually look at the user.

There is a small third cluster that is beginning to fill this gap. Computer vision-based scoring of exercise form is used in Onyx, Kemtai and Tempo. It is coupled to a hardware mirror by Tempo; Onyx and Kemtai are app-only. None of them now encases the form-scoring layer within a coach-trainee social layer that enables a real coach to look at what the AI looked at. That is the gap that this project is aimed to fulfill.

## 2.2 Literature Review

The following table summarises the works that directly informed this project. They span four areas: pose estimation models, web-based real-time inference, web architecture and security standards, and the software-quality and usability literature against which the build was evaluated. Each row cites a numbered reference in the References section at the end of the report; numbers in brackets follow IEEE format.

**Table 2.1: Summary of Literature Reviewed**

| Ref. | Author(s) / Source | Year | Relevance to this project |
|---|---|---|---|
| [1] | Bazarevsky et al. (BlazePose, arXiv) | 2020 | Defines the on-device 33-keypoint pose model that MediaPipe Pose ships. The 25–30 frames-per-second target in this project comes from BlazePose's reported latency on mobile CPUs. |
| [2] | Cao et al. (OpenPose, IEEE TPAMI) | 2021 | Part-affinity-fields multi-person pose estimation. Used as the accuracy benchmark in later browser-based pipelines. |
| [3] | Lugaresi et al. (MediaPipe, arXiv) | 2019 | The MediaPipe framework paper. Describes the calculator-graph and WASM execution model that the live-workout module depends on. |
| [4] | Grishchenko & Bazarevsky (Google AI Blog: BlazePose GHUM) | 2020 | Describes the GHUM-based 3D world-landmark output that the squat detector uses for the bilateral knee-distance heuristic. |
| [5] | Howard et al. (MobileNetV3, ICCV) | 2019 | The mobile-class CNN backbone family that BlazePose builds on; explains the latency-accuracy trade-off the project relies on. |
| [6] | Tan & Le (EfficientNet, ICML) | 2019 | Compound-scaling reference for understanding model-size trade-offs; informed the decision to keep MediaPipe over a larger TensorFlow.js model. |
| [7] | Zhang et al. (MediaPipe Hands, arXiv) | 2020 | Companion model to BlazePose using the same MediaPipe pipeline; cited for the framework-level performance claims. |
| [8] | Yan et al. (ST-GCN, AAAI) | 2018 | Skeleton-based action recognition with graph convolutions; informed the choice of state-machine over learned classifier given limited training data. |
| [9] | Shi et al. (2s-AGCN, CVPR) | 2019 | Two-stream adaptive graph convolutional network; further evidence that classifier-based exercise recognition needs significant data, supporting the lighter heuristic approach used here. |

| Ref. | Author(s) / Source | Year | Relevance to this project |
|---|---|---|---|
| [10] | Fielding & Taylor (REST, ACM TOIT) | 2002 | Defines the architectural style of the JSON-over-HTTP API surface; informs the OpenAPI specification produced at /api/openapi. |
| [11] | Jones, Bradley & Sakimura (RFC 7519: JWT) | 2015 | Specifies the JWT format used for the role-aware authentication cookie. |
| [12] | Rescorla (RFC 8446: TLS 1.3) | 2018 | Specifies the secure transport layer used by every protected endpoint. |
| [13] | Cooper et al. (RFC 5280: X.509) | 2008 | Certificate profile used by TLS to authenticate the server. |
| [14] | Eastlake & Hansen (RFC 6234: SHA) | 2011 | Hash primitive underlying JWT signatures and verification-token storage. |
| [15] | Fielding et al. (RFC 9110: HTTP Semantics) | 2022 | Modern HTTP semantics used throughout the API surface. |
| [16] | OpenAPI Initiative (OAS 3.1.0) | 2021 | Specification format produced by the API documentation generator. |
| [17] | OWASP Foundation (Top Ten) | 2021 | Threat-model checklist applied to authentication, rate limiting, and input validation. |
| [18] | Provos & Mazières (bcrypt, USENIX) | 1999 | Password-hashing scheme used for credential storage. |
| [19] | ISO/IEC 25010 (SQuaRE quality model) | 2011 | Source of the non-functional requirements taxonomy in Chapter 3 (usability, performance, security, maintainability, portability, reliability). |
| [20] | ISO/IEC/IEEE 29148 (Requirements Engineering) | 2018 | Modern successor to IEEE 830-1998; provides the structure used for the functional requirements catalogue. |
| [21] | IEEE Std 1016 (SDD) | 2009 | Software design description standard the architecture chapter follows. |
| [22] | ISO/IEC/IEEE 12207 (Software Life Cycle) | 2017 | Life-cycle processes that the waterfall-iterative hybrid in this project maps onto. |
| [23] | Nielsen (Usability Engineering) | 1994 | Source of the ten heuristics applied to the role-specific dashboard designs. |

| Ref. | Author(s) / Source | Year | Relevance to this project |
|---|---|---|---|
| [24] | Norman (Design of Everyday Things) | 2013 | Affordance and feedback principles applied to the live-workout UI. |
| [25] | Shneiderman et al. (Designing the UI, 6th ed.) | 2016 | Eight golden rules of interface design used as a secondary review checklist. |
| [26] | Google LLC (MediaPipe Pose Landmarker docs) | 2024 | Production API used by the live-workout module. |
| [27] | MongoDB Inc. (Manual v7) | 2024 | Document-store reference; TTL indexes used here for verification tokens and rate-limit buckets. |
| [28] | Vercel (Next.js 14 App Router Docs) | 2024 | Routing, middleware, and API-route execution model the system depends on. |
| [29] | Stripe (Subscriptions Integration Docs) | 2024 | Specifies the Checkout-Session and webhook signature flow implemented in the billing module. |
| [30] | OpenAI (Chat Completions API Reference) | 2024 | Request/response shape that the AI conversational coach endpoint adheres to. |
| [31] | W3C (MediaCapture / Streams) | 2023 | Browser standard governing camera access, frame rate, and constraints used by the live workout. |
| [32] | Ecma International (ECMAScript 2023) | 2023 | Language specification for the JavaScript/TypeScript runtime. |
| [33] | WHO (Physical Activity Guidelines) | 2020 | Public-health context cited in motivation and impact sections. |

### 2.2.2 Technical Background and Stack Choices

Pose estimation. MediaPipe Pose Landmarker [26], the productionised form of the BlazePose model [1], [4] is used in the live-workout module. The GHUM body model produces a 33-keypoint metric-space output at the body model's burn point [4], which is a 33-keypoint normalised image-space keypoint output. It is the world output of the squat detector which makes the bilateral knee-distance check of the squat detector reliable across camera angles since image-space coordinates collapse the depth axis. Previous systems like OpenPose [2] had identified the multi-person regression-and-affinity-fields approach, although they needed hardware suitable by the standards of a graphics card to run in real time and thus was unsuitable to a browser-only deployment. The paper MediaPipe framework [3] outlines the calculator-graph architecture and WebAssembly compilation strategy

that enables the same model to be executed on CPU at 25 to 30 frames per second on a 2019-class laptop. The reference to the backbone-architecture that contextualises the choice of model-size is that of BlazePose which uses a MobileNetV3-class encoder, which trades a small margin of accuracy with a large margin of latency benefit over larger backbones. MediaPipe Hands [7] is a sibling model that uses the same framework, and provides a handy baseline of what to expect.

Exercise classification approach. Two skeleton based action recognition references were considered to classify learned exercises, based on ST-GCN [8] and 2s-AGCN [9]. They are both based on graph convolutions on keypoint trajectories and need thousands of labelled clips per exercise. Learned classification was, however, rejected in favour of a per-exercise hand-coded state machine, in which joint angles and visibility scores are used as transition functions. These two papers are mentioned here to support the decision and not to say that it is directly used.

Web architecture. The application is based on the architectural style of a REST described by Fielding and Taylor [10]. All resources have fixed URLs, methods are represented by the HTTP verbs which are governed by RFC 9110 [15] and payloads are in the form of JSON. These same Zod schemas, at runtime to validate incoming requests, are walked at build time to emit an OpenAPI 3.1 specification [16] served at /api/openapi. The application is based on the Next.js 14 framework that includes the App Router [28] that places client UI and server-side route handlers in the same TypeScript project. The Edge runtime restriction that was encountered during development that jsonwebtoken cannot be executed as there since Edge has no Node crypto is documented in [28] and fixed here by only performing cookie-presence checks in middleware, leaving the actual signature verification to be done by the Node-runtime API routes.

Security and identity. Authentication is based on JSON Web Tokens as specified by RFC 7519 [11]. All tokens are signed using HMAC-SHA-256 and contain a small payload: user id, email, role. RFC 6234 [14] specifies the hash primitive. Tokens are carried in HTTP-only cookies; the certificate profile is defined by RFC 5280 [13]. The OWASP Top Ten threat model [17] is followed by rate limiting, validation and authorization. Before being stored, passwords are hashed using bcrypt [18] with cost factor 10. SHA-256 digests containing verification and password-reset tokens are stored in MongoDB [27] as SHA-256 digests with a TTL index in MongoDB.

Data store. Persistence MongoDB [27] accessed using Mongoose. The collections are users, workoutsessions, workoutplans, exercises, coachclients, coachnotes, messages, verificationtokens, ratelimitbuckets and auditlogs. Two TTL indexes one on verificationtokens.expiresAt and one on ratelimitbuckets.expiresAt — automatically garbage-collect short-lived data without application code. Schema

flexibility prevailed: the User document was modified to add coachApplication, coachProfile, subscription and emailVerified subdocuments in the middle of the project, something a relational schema would have forced migrations to add.

Billing and AI services. The subscription primitive is offered by Stripe [29]. The application sets up Checkout Sessions of upgrade and Customer Portal of management; the entitlement state is updated by a signature-verified webhook on the event customer.subscription.created, .updated, and . deleted. The OpenAI Chat Completions API [30] backs the AI conversational coach behind a Pro-tier gate. Both integrations adhere to the documented request/response shapes word-for-word, and no transformation that may drift across versions.

Software-engineering standards. Requirements follow ISO/IEC/IEEE 29148:2018 [20], the modern successor to IEEE 830-1998. Design documentation is based upon IEEE Std 1016-2009 [21]. The quality attributes (usability, performance, security, maintainability, portability, reliability) are borrowed by ISO/IEC 25010:2011 [19]. The hybrid life cycle of the waterfall-iterative system is mapped to the ISO/IEC/IEEE 12207:2017 [22]. The usability heuristics applied to the dashboards are the ones of Nielsen [23], further elaborated with the affordance and feedback principles of Norman [24] and the eight golden rules of Shneiderman et al. [25]. The MediaCapture/Streams W3C specification [31] is used to govern the browser camera access pattern in the live-workout module. The runtime language is ECMAScript 2023 [32].

### 2.2.1 Comparable Applications

The following table compares five products that overlap in scope. The features column is restricted to capabilities that this project either matches or deliberately differs from.

**Table 2.2: Comparable Applications**

| Application | Region | Key features |
|---|---|---|
| Nike Training Club | Global | Guided video workouts, tap-based tracking, no form scoring, no human coach layer, free with paid premium. |
| Onyx | Global | Browser-based AI form scoring for body-weight exercises, no human coach connection, subscription model. |
| Kemtai | Global/Israel | Pose-based exercise scoring positioned for clinical and rehab use, B2B partnerships rather than direct consumer. |
| Tempo | United States | Hardware mirror with depth-sensing camera, AI form scoring, premium hardware price point. |

| Application | Region | Key features |
|---|---|---|
| MyCoach (BD) | Bangladesh | Local marketplace connecting trainers to clients; no AI form analysis, scheduling and payment focused. |

## 2.3 Gap Analysis

The above comparison indicates that there are three patterns. First, the apps that have AI form scoring (Onyx, Kemtai, Tempo) do not allow a human coach to see the AI output and react to it. Second, AI form scoring is not present in the platforms that include human coaches (MyCoach, in-app coaching marketplaces); the coach is not visible to the trainee between sessions. Third, no product surveyed caters to the small-coaching-business case in Bangladesh - independent coaches who desire a lightweight platform to retain clients within a single place. FitFlow fits perfectly in the overlap where these three gaps meet.

**Table 2.3: Gap Analysis**

| Feature | Nike T.C. | Onyx | Kemtai | Tempo | MyCoach | FitFlow |
|---|---|---|---|---|---|---|
| On-device CV form scoring | No | Yes | Yes | Yes (HW) | No | Yes |
| No video uploaded to server | N/A | Yes | Yes | No | N/A | Yes |
| Coach can review trainee form data | No | No | No | No | Partial | Yes |
| Coach application + admin review | No | No | No | No | Yes | Yes |
| Direct coach ↔ trainee messaging | No | No | No | No | Yes | Yes |
| Trainee 30-day progress dashboard | Limited | Yes | Yes | Yes | No | Yes |
| AI conversational coach | No | No | No | Yes | No | Yes (Pro) |
| Free entry tier | Yes | No | B2B | No | Yes | Yes |

## 2.4 Summary

This chapter framed the problem space. Existing free fitness apps do not score form. AI-form-scoring apps do not connect a real coach. Coach marketplaces do not surface the data that would make the coach more effective. FitFlow is designed to occupy the intersection: on-device computer vision form scoring, persisted as session history, made visible to a real coach inside a directed coach-client relationship. Chapter 3 turns this positioning into requirements and a design.

# Chapter 3

# Research Methodology

It is through this chapter that FitFlow was specified, designed and constructed. It starts with requirements, takes a stroll through the system design at three levels of data-flow detail, and finishes with the project plan and task allocation. The idea is to leave a reader who has not seen the application with sufficient information to know why each of the modules exists, and how they fit.

## 3.1 Requirement Analysis & Design Specification

### Functional Requirements

The functional requirements were classified according to the user role.

### Trainee.

Register and log in; either at sign-up, choose either I want to train or I want to coach; or, after the sign-up, choose to create a pending application where you submit a written bio of at least thirty characters.

Do a live workout of squat, push-up, or plank with counting of repetitions and the smoothed form score; the resulting session is only sustained when at least five seconds long.

View progress over the next thirty days on the dashboard: current streak, number of sessions, average form, total number of reps, reps-per-day bar chart, form-score trend sparkline, per-exercise breakdown.

Search the list of proven coaches; review the profile of a coach with bio; send a request to be connected with a coach; cancel a request.

After connecting, send and receive direct messages with the coach and get a badge on the navigation-bar on new messages that indicates unread.

Email reset password and confirm email by a token-link flow.

### Coach

Upon login, a coach-specific dashboard displays land, today's clients, recent sessions with all clients, a list of clients not active within seven days and a bio-completeness index.

Edit the public profile bio (301000 characters); preview the public profile as the editor.

Accept or reject incoming connection requests with one click; an active connection opens up messaging and client detail view.

Open a per-client detail page, displaying recent sessions, a form-score trend sparkline, and a private notes log; add new notes not visible to the client.

Send and receive messages with connected customers using a two pane interface with thread listing and conversation view.

### Administrator

Admin dashboard with hero statistics (number of members, number of sessions in the last seven days, number of pending coach applications) and detail tiles (coaches, pending coaches, suspended users, workout plans).

Search and paginate user list; filter by role; promote or demote any user except own self; suspend or unsuspend any user except own self.

Check the queue of coach applications; approve (which changes the role to coach and copies the coach application bio to the public profile) or reject (which requires a written justification and notifies the coach application applicant by email).

Any change of roles and any suspensions are documented in an audit log to be accountable.

### Non-Functional Requirements

Usability. The interface must be usable by a fitness user with no technical background; the role specific dashboards must give it a sense of what a user can do next.

Performance. The interactions of the page are expected to respond within less than 300 ms on the dashboard, and less than 100 ms after the local pose model has been warmed, pose detection is expected to maintain at least 25 frames per second on a 2019-class laptop.

Security. There should be sensitive operations that require valid JWTs; tokens should carry the role to prevent re-fetching with every request; rate limiting must apply to login and registration endpoints, as well as password reset and verification endpoints.

Privacy. Frames taken by the camera should not go beyond the device. The coach notes should not be seen by the client. Any notes left by a reviewer on rejected applications should only be visible to the applicant and the reviewer who wrote the note.

Scalability. The data model must not need restructuring to support a hundredfold increase in the number of users; the rate limiter must be able to work with multiple Node instances.

Maintainability. The code must be type-checked all the way through; the API surface must be documented using an OpenAPI specification generated using the same Zod schemas used to run-time validate.

### 3.1.1 Overview

FitFlow is a Next.js 14 application [28] with the App Router. The codebase is identical to the React client as well as the API routes. The database can be easily hosted on Atlas in production and does not require any modification to the application to run on MongoDB [27]. The authentication is JWT-based [11], where the token is stored in an HTTP-only cookie and the role of the user is embedded in the payload so middleware can do role-aware routing without an additional database read.

The three top level surfaces are associated with the three roles. The trainees come to /dashboard. Coaches arrive at /coach. Administrators touch down at /admin. The one source of truth is a shared utility (homeFor in src/lib/roleHome.ts) which contains the information of which path each role considers home; the login route, registration route, and dashboard route all consult this source of truth to ensure the redirects never drift.

Live workout module is the only application component which renders heavy work in the browser. MediaPipe Pose [1], [26] is run at the camera feed at the device-native frame rate; the resulting landmarks are fed to per-exercise detectors which update internal state. The React layer is exposed to a small set of metrics (rep count, form score, calories estimate, duration, tempo) via a callback. No information in the video is posted to the server. The only aggregate measures POSTed to /api/workout-sessions when the user ends the workout are the final aggregate measures.

### 3.1.2 Proposed Methodology / System Design.

A waterfall-iterative hybrid as defined in section 1.4 has been used to build the system. The high-level design flow, shown in figure 3.1, is requirements into architecture, architecture into module design, and the module design into the four parallel implementation tracks (UI, persistence, computer vision, role layer). Implementation gives, testing into deployment, deployment into testing, and testing into implementation.

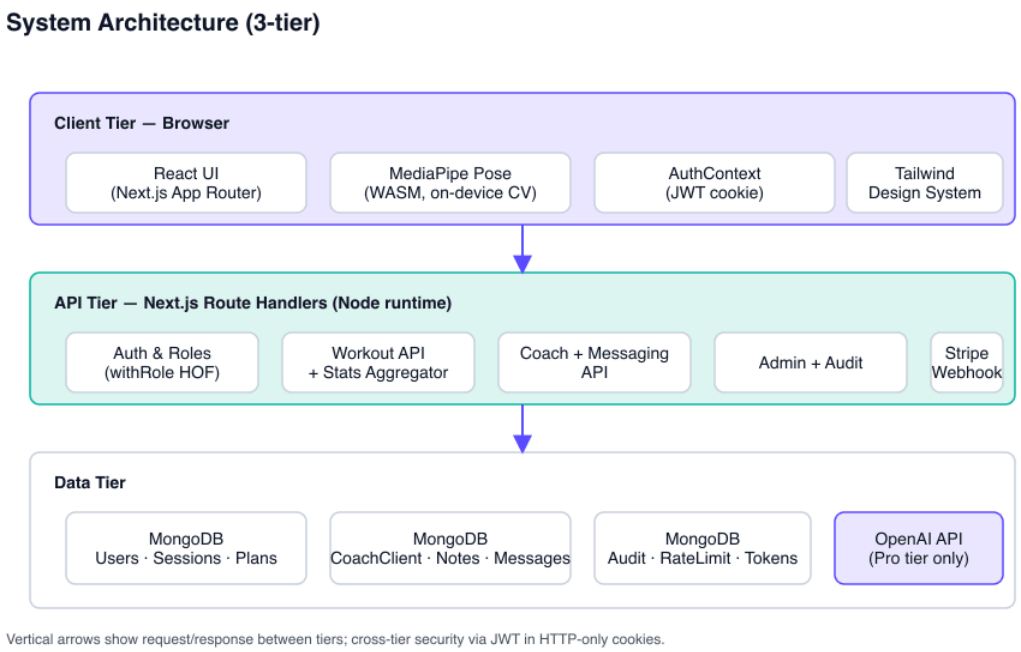



*Figure 3.1: System Architecture (3-tier).*

FitFlow has major architectural modules:

- Role layer and authentication. Email/ password authentication using bcrypt-hashed passwords, JWT tokens with role, and higher-order function withRole(roles, handler) that gates API routes by role.
- Trainee module. Dashboard, profile, live workout, calculators, coach directory, billing.
- Coach module. Overview, clients, client detail with notes and trend, requests, profile editor, messaging.
- Admin module. Introduction, list of users, review of coach-application.
- Computer vision module. PoseDetector wrapper of MediaPipe, and three exercise-specific state machines (squat, push-up, plank).
- Messaging module. Single Message collection keyed on (coachId, clientId) with polling-based delivery, and navigation-bar badge unread.
- Billing module. Stripe checkout, customer portal, signature-verified webhook, entitlement helper which always reads the current subscription in the database such that upgrades are always applied without requiring re-login.

### 3.1.3 Functional and Non-Functional Requirements

The full enumerations are listed at the start of section 3.1. Non-functional requirements draw on ISO/IEC 25010:2011 [19] and on the OWASP Top Ten threat model [17].

### 3.1.4 Data Flow Diagram

Three levels of DFD describe the system at increasing detail.

**Diagram Symbols and Notation**

| Name | Description |
|---|---|
| Process | A transformation of input data into output data; drawn as a rounded rectangle. |
| External Entity | A person or system outside the system boundary that interacts with it; drawn as a rectangle. |
| Data Store | A repository where data is stored; drawn as an open rectangle. |
| Data Flow | A directional arrow showing the movement of data between processes, stores, and external entities. |

### DFD Level-0 (Context Diagram)

The Level-0 diagram treats FitFlow as a single process and shows its three external entities — Trainee, Coach, Admin — and the high-level flows between them and the system. The Trainee submits workout sessions and reads progress. The Coach submits notes and messages and reads client data. The Admin manages users and reviews applications. All three flows enter and leave through the API layer.

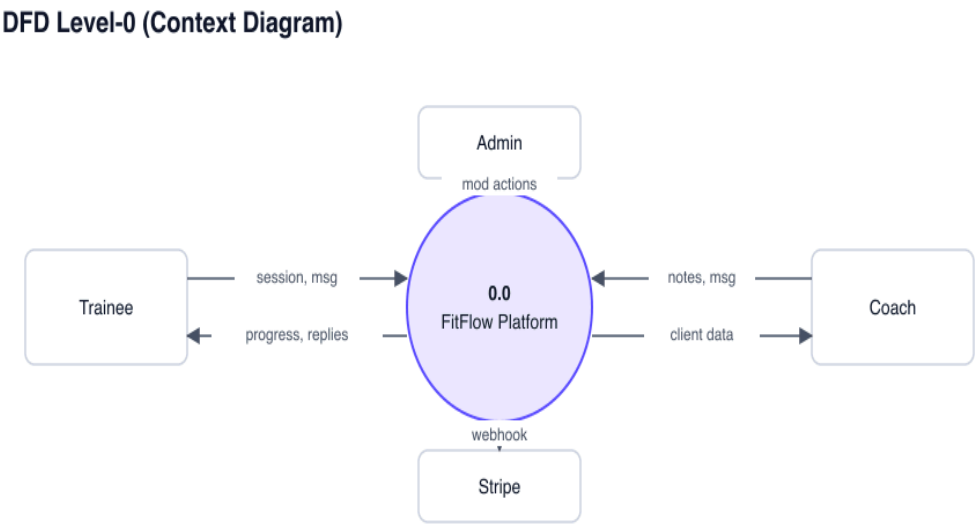



*Figure 3.2: DFD Level-0 (Context Diagram).*

## DFD Level-1 (Major Processes)

Level-1 The single process is broken down into seven major ones which are identified as corresponding to the architectural modules:

9. 1.1 Authentication. Register, log in, verify email, reset password.
10. 1.2 Workout session. Live workout run, persist session, fetch history.
11. 1.3 Progress aggregation. Compute thirty-day series, streak, totals.
12. 1.4 Coach link. Request write, accept, decline, withdraw.
13. 1.5 Coach review. Read client sessions, write on personal notes.
14. 1.6 Messaging. Send, fetch, mark-read.
15. 1.7 Administration. Add users, alter role, suspend, peruse applications.

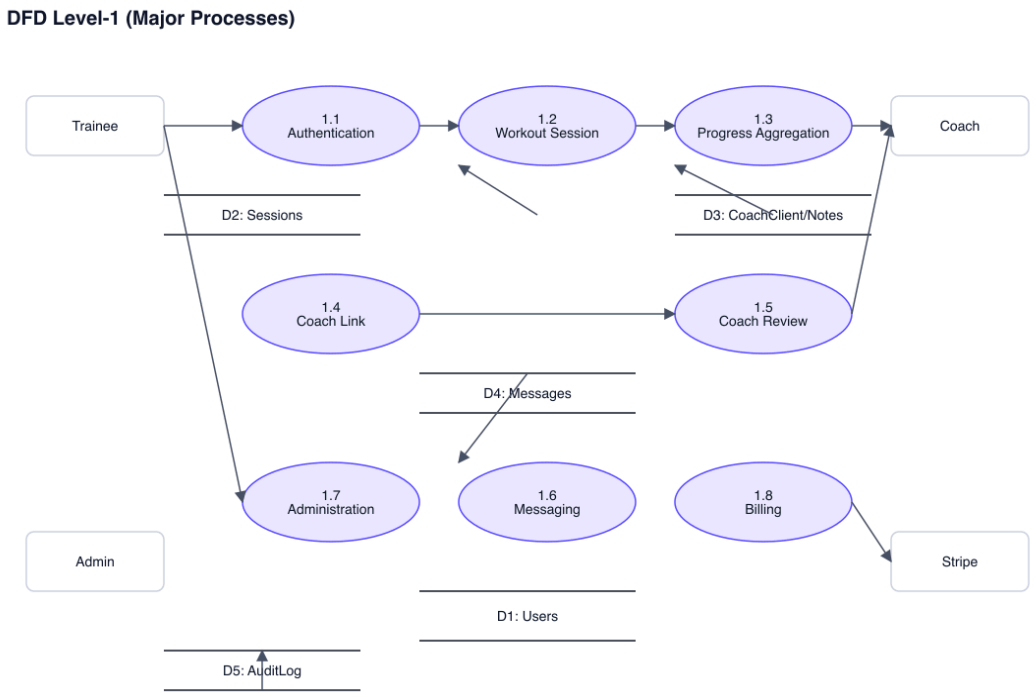



*Figure 3.3: DFD Level-1 (Major Processes).*

## DFD Level-2 (Live Workout Pipeline)

The Live Workout process (1.2) is further broken down since it is the technical heart of the project. The Level-2 diagram illustrates the six processes in the company:

16. 2.1 Camera capture. Get video frames, at the rate of the device-native rate, of the user web camera.
17. 2.2 Pose detection. Input each frame to MediaPipe Pose and obtain 33 visible landmarks with scores.
18. 2.3 Exercise classification. Stream the landmark stream to the active exercise detector according to the choice of the user.
19. 2.4 State-machine update. Each detector has a rep state (idle, ready, down, up) which is updated when the threshold is crossed.
20. 2.5 Form analysis. Calculate angle-based form penalties, soften it using an exponential decay, uncover the outcome in a 0-100 form score.
21. 2.6 Metric persistence. On session stop POST aggregated measures to /api/workout-sessions that writes a document of WorkoutSession.

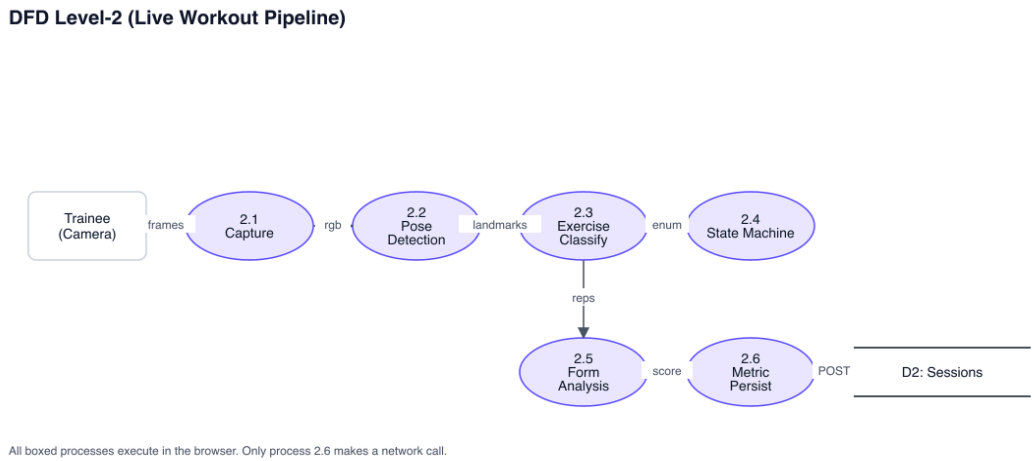



*Figure 3.4: DFD Level-2 (Live Workout Pipeline).*

## Use Case Diagram

Figure 3.6 enumerates the use cases visible to each actor. Trainee actor: register, log in, run workout, view progress, browse coaches, send coach request, message coach, manage subscription. Coach actor: review applications inbox, accept or decline request, view client list, view client detail and trends, write private note, message client, edit public bio. Admin actor: list users, change role, suspend or unsuspend, review coach application, approve, reject, view audit log.

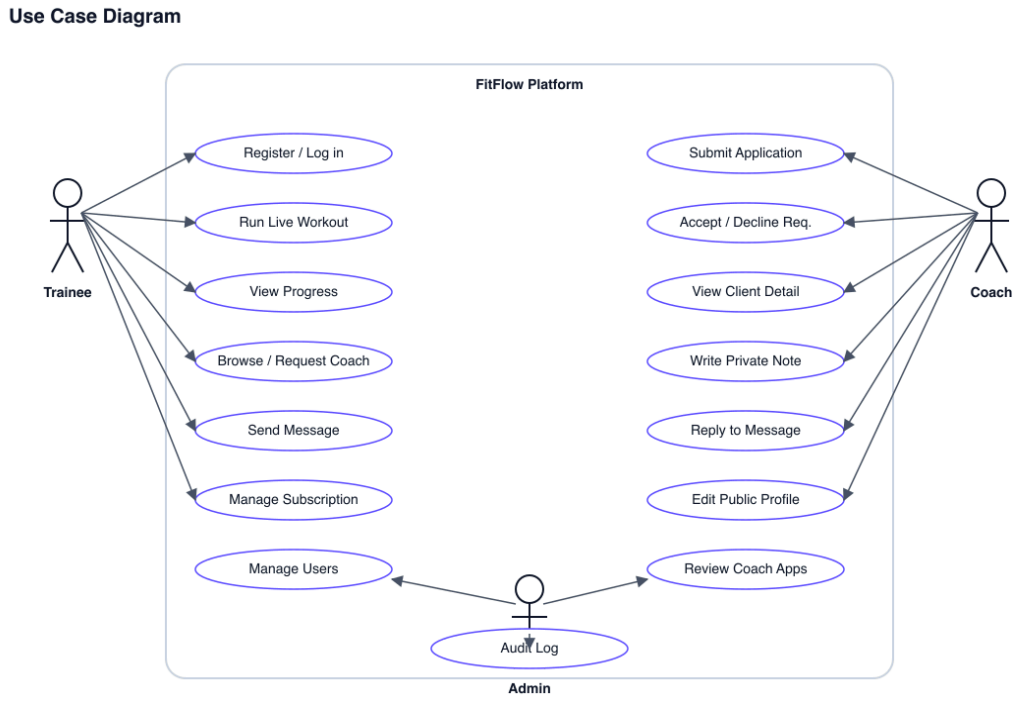



*Figure 3.5: Use Case Diagram.*

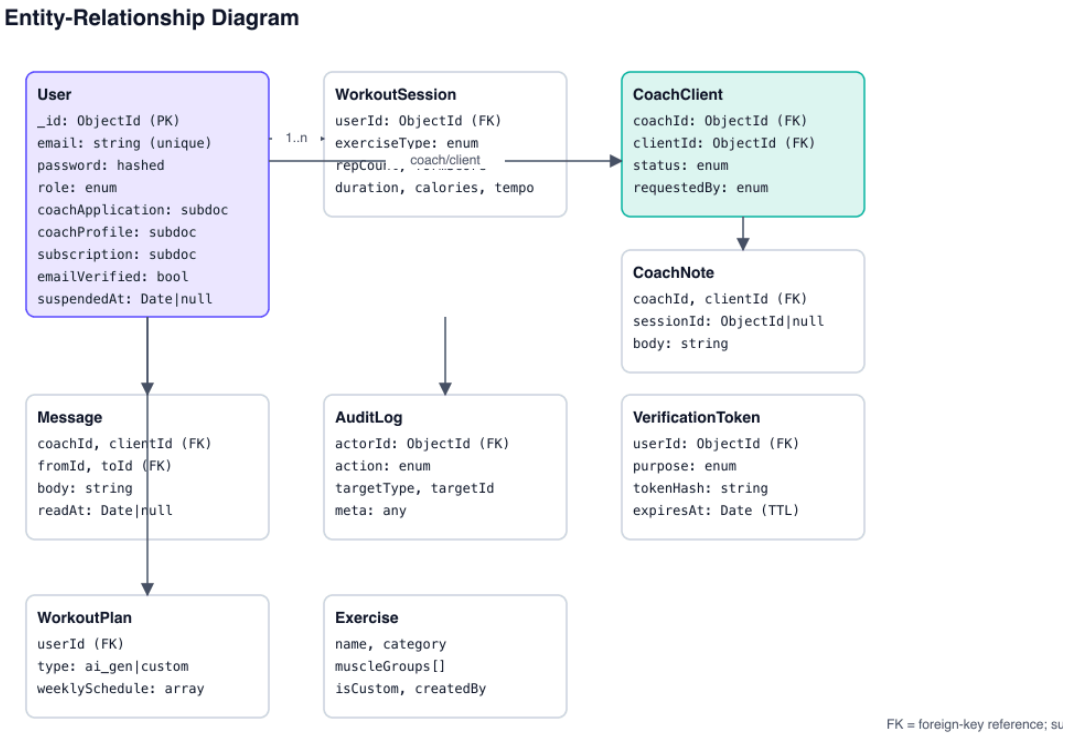



*Figure 3.6: Entity-Relationship Diagram.*

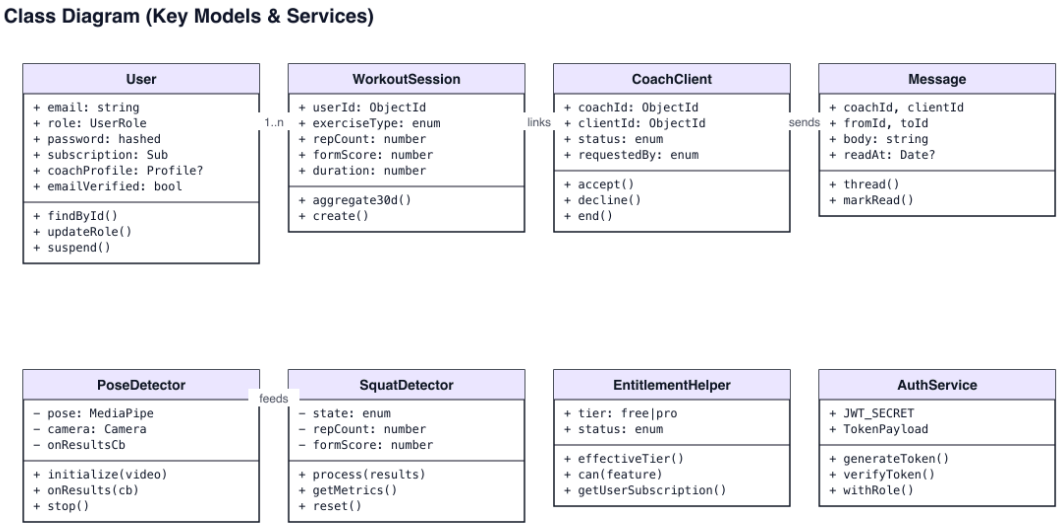



*Figure 3.7: Class Diagram (Key Models and Services).*

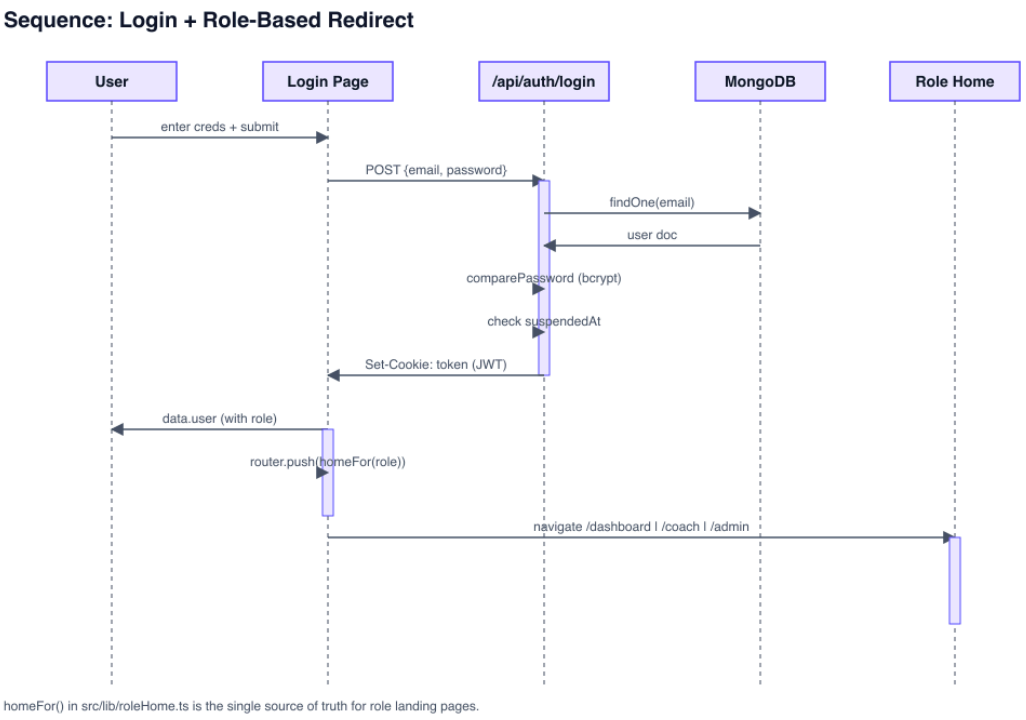



*Figure 3.8: Sequence Diagram — Login + Role-Based Redirect.*

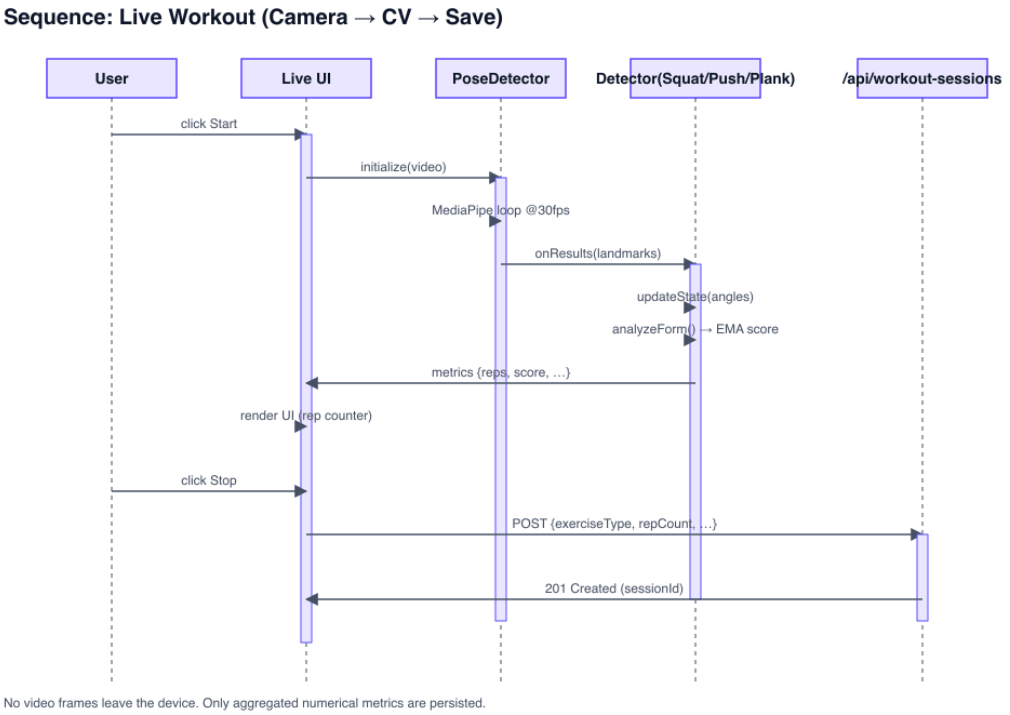



*Figure 3.9: Sequence Diagram — Live Workout Pipeline.*

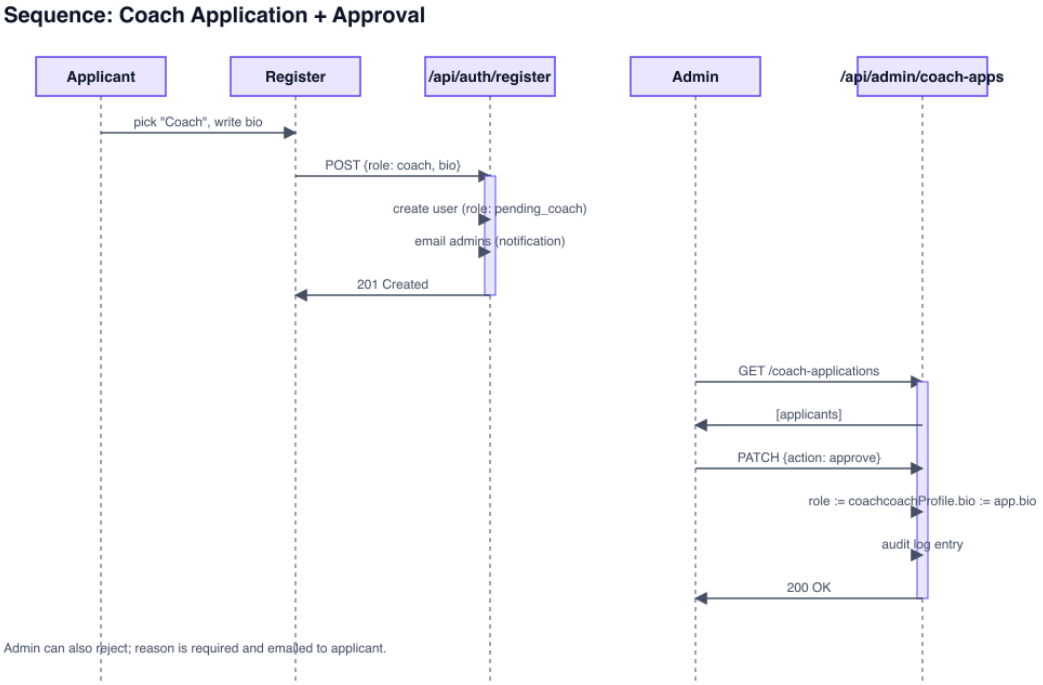



*Figure 3.10: Sequence Diagram — Coach Application + Approval.*

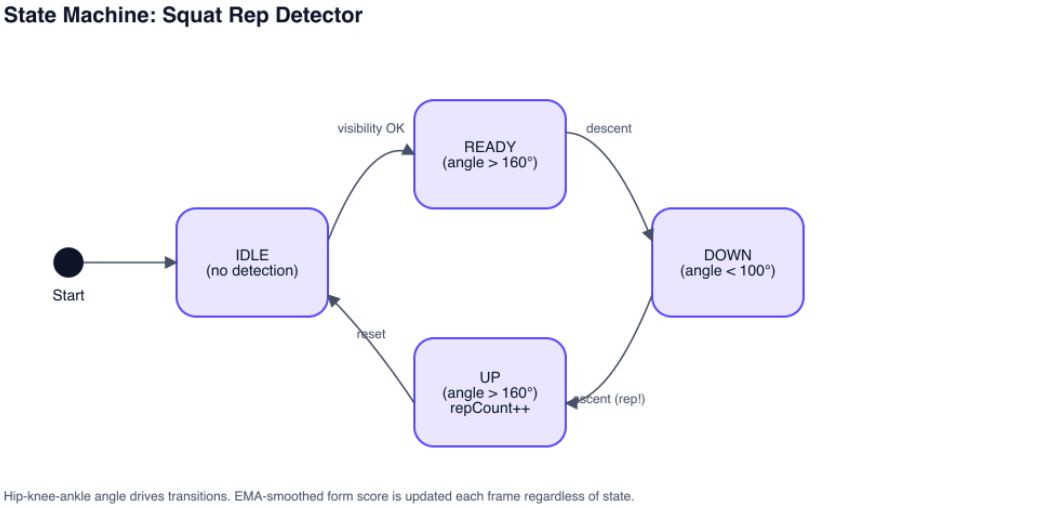



*Figure 3.11: State Machine — Squat Rep Detector.*

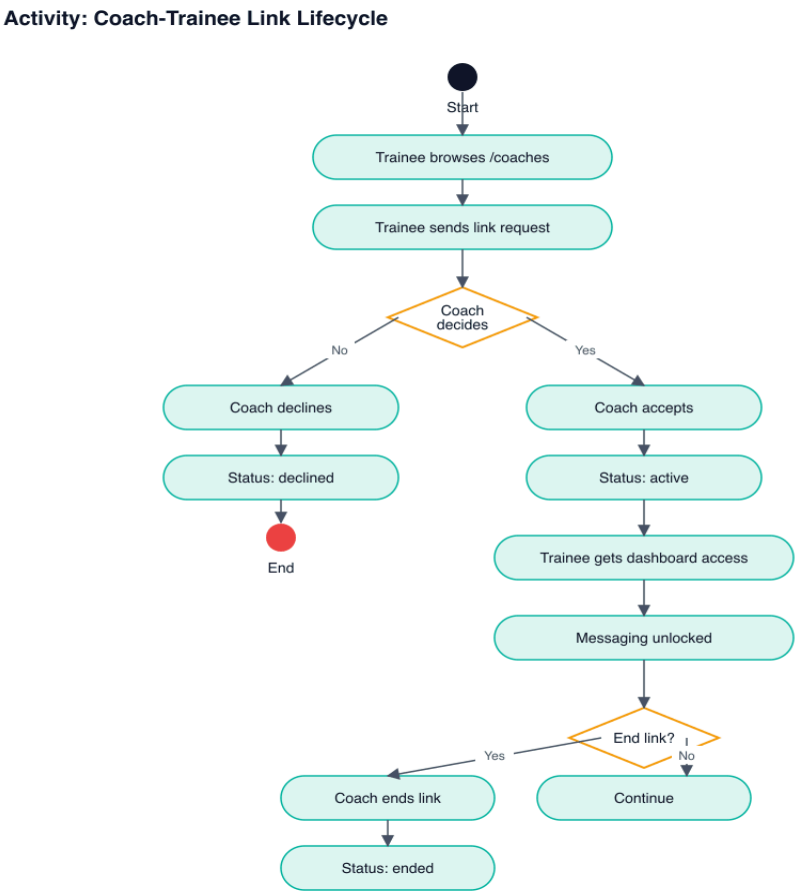



*Figure 3.12: Activity Diagram — Coach-Trainee Link Lifecycle.*

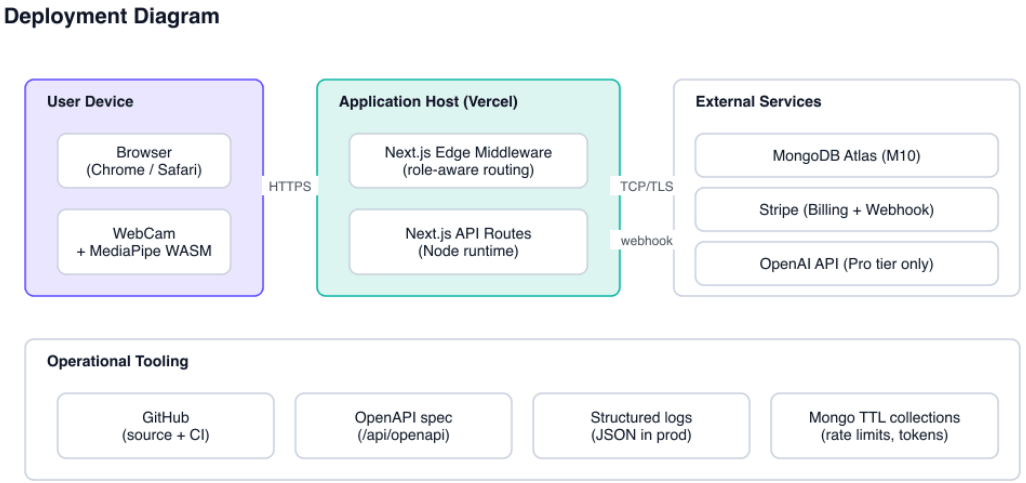
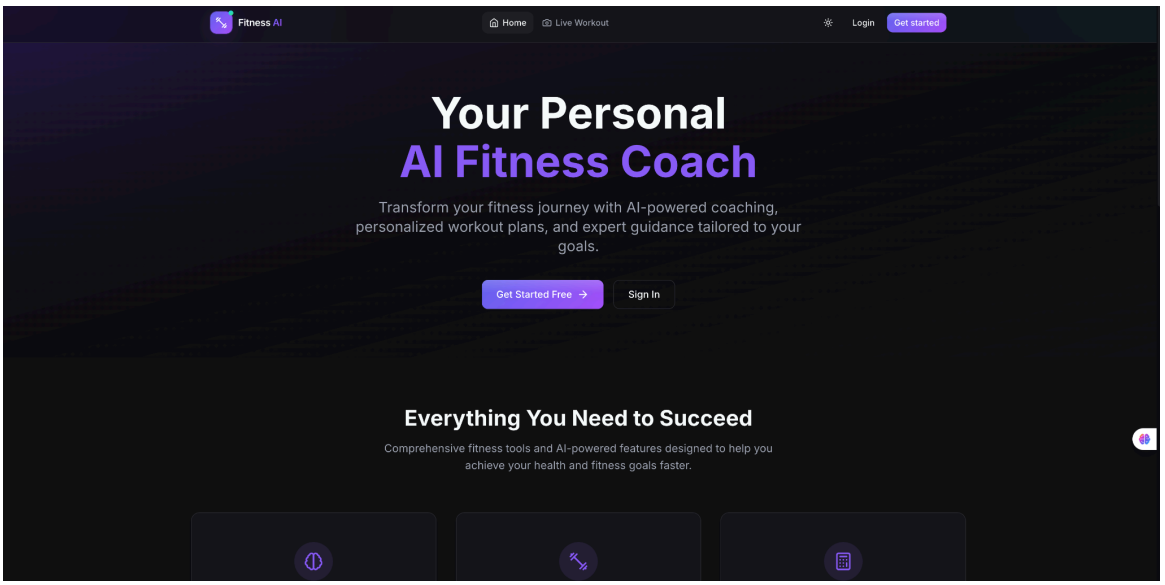
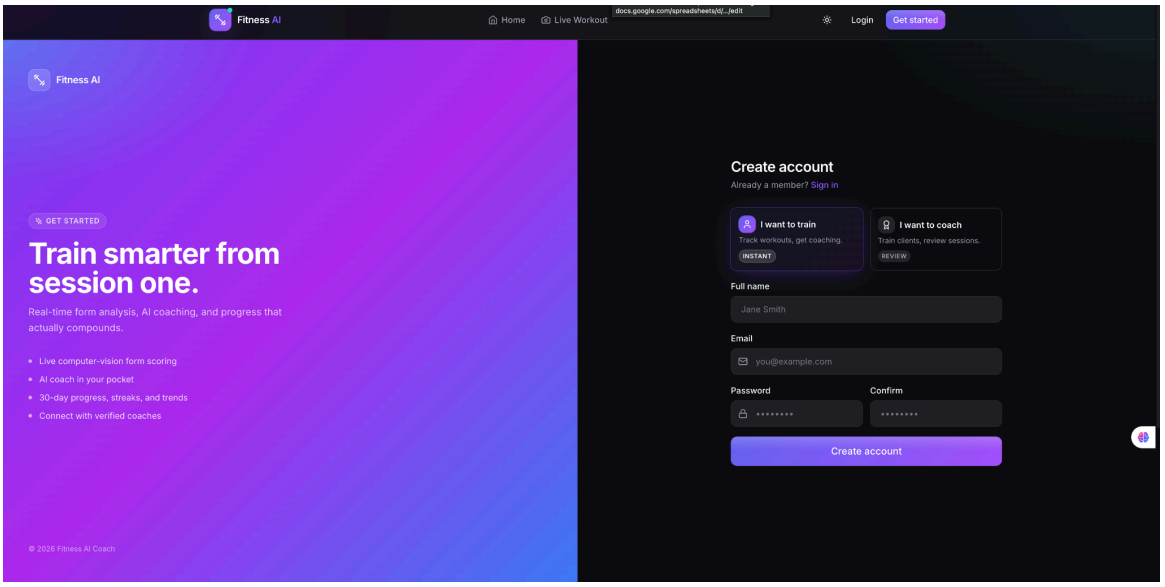


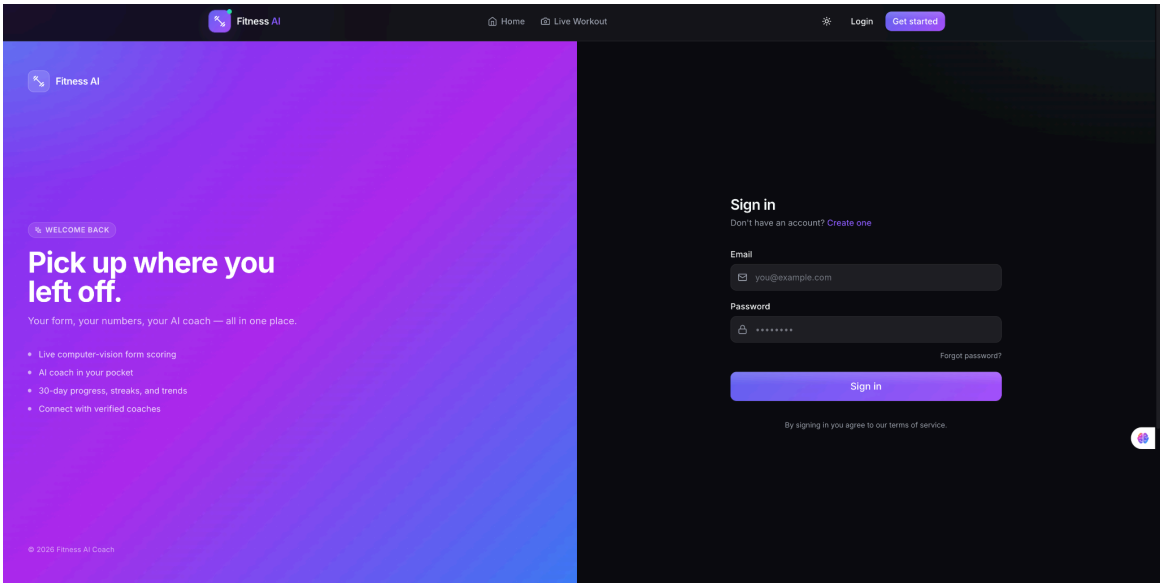

*Figure 3.13: Deployment Diagram.*

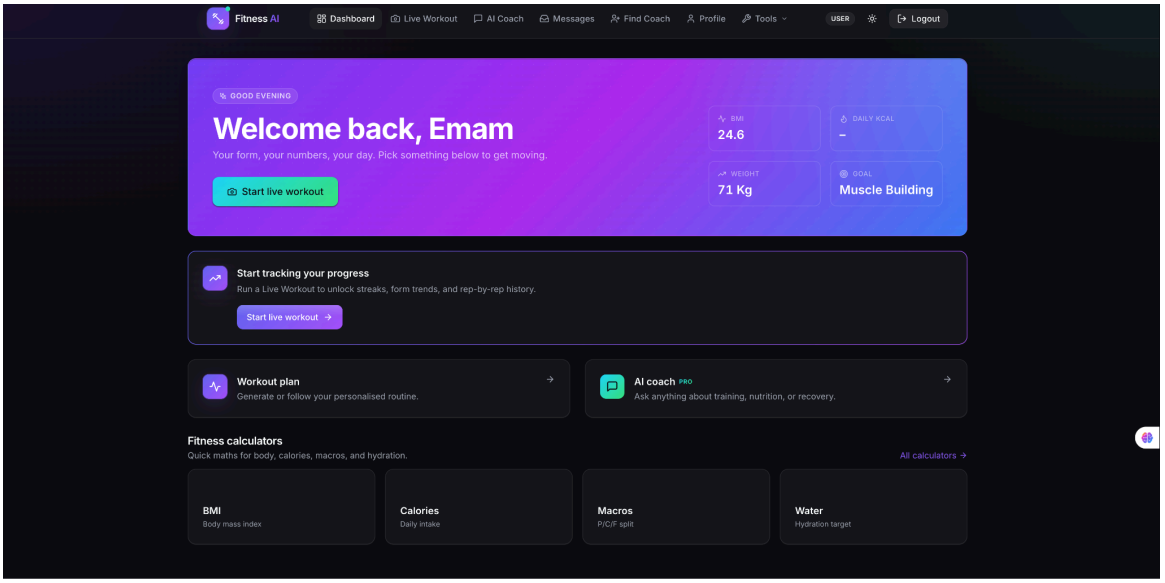
### 3.1.5 UI Design

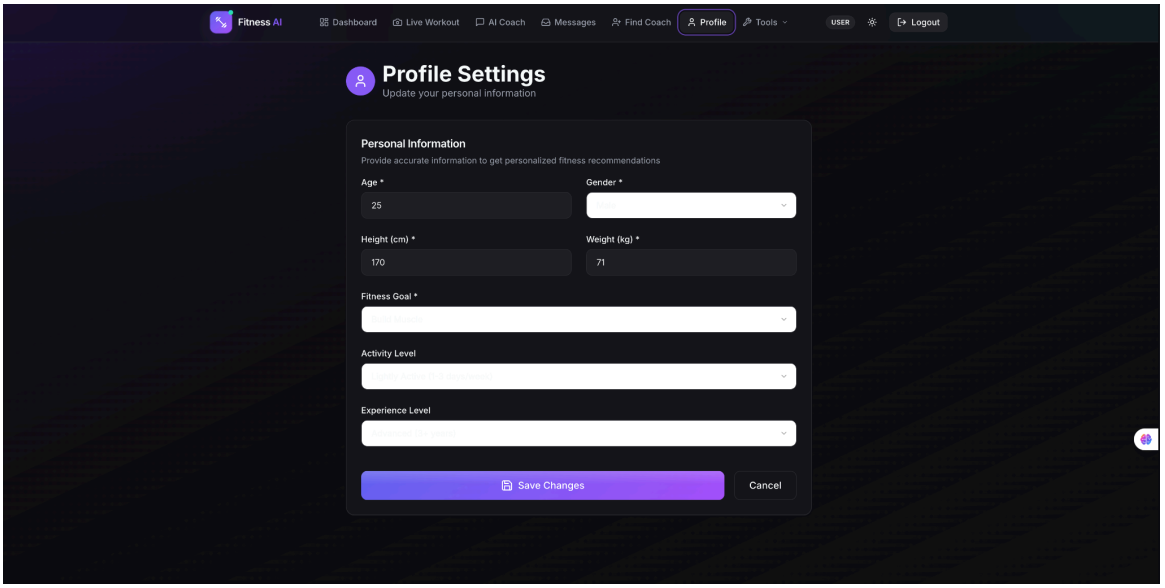
The interface uses a custom design system built on Tailwind CSS. The primary colour is an indigo-violet gradient; the accent colour is a cyan-mint gradient that signals workout-active states and the coach role.

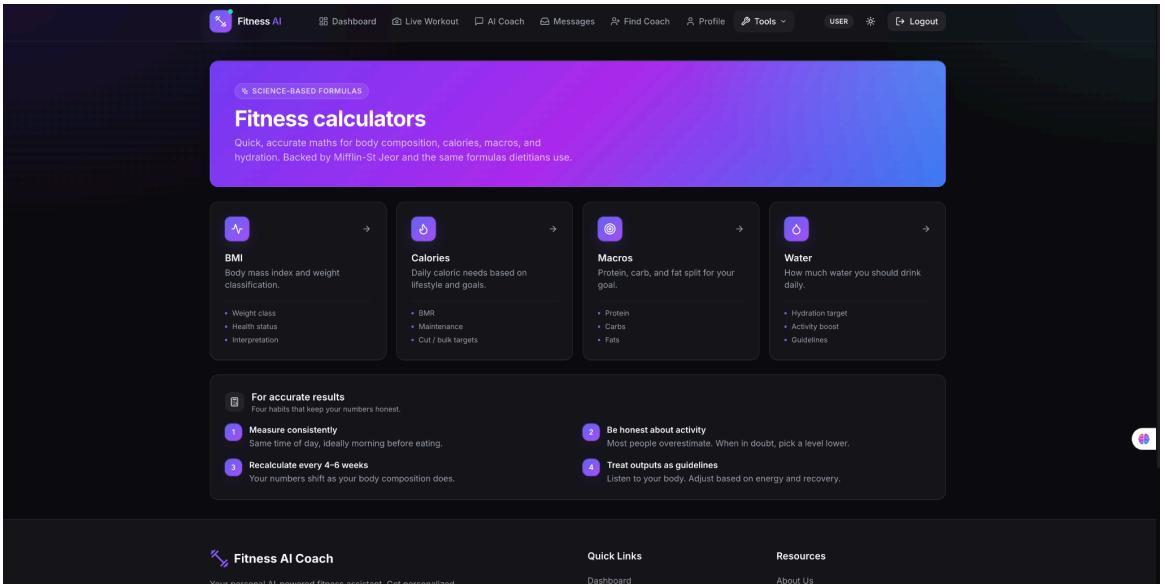

Fitness AI
Home
Live Workout
Login
Get started
Your Personal
AI Fitness Coach
Transform your fitness journey with AI-powered coaching, personalized workout plans, and expert guidance tailored to your goals.
Get Started Free →
Sign In
Everything You Need to Succeed
Comprehensive fitness tools and AI-powered features designed to help you achieve your health and fitness goals faster.

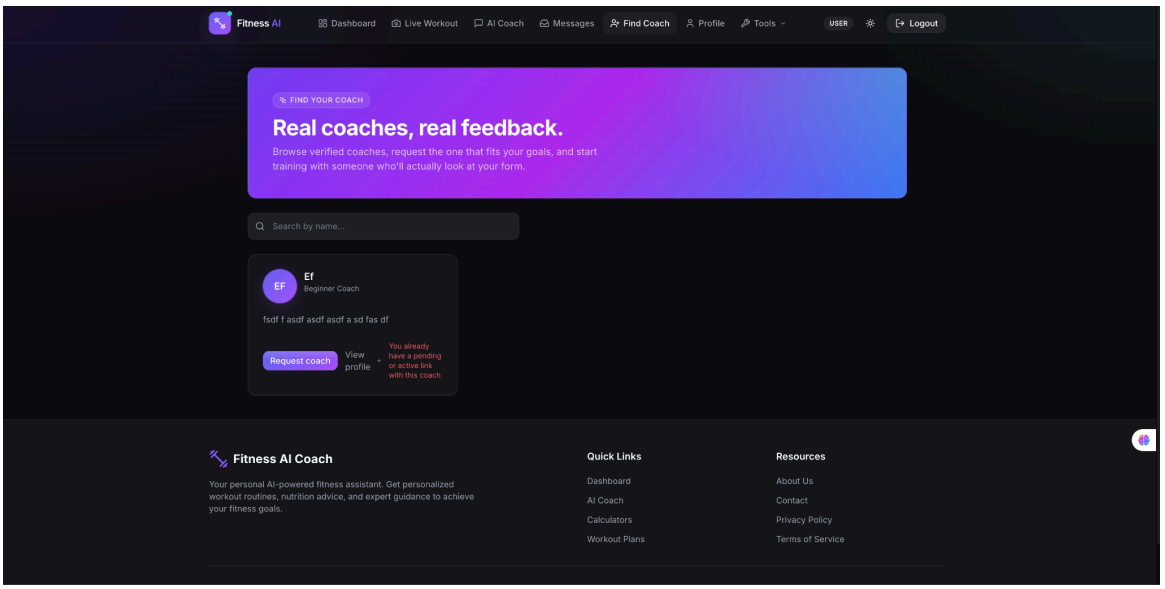

Fitness AI
Home
Live Workout
Login
Get started
Fitness AI
GET STARTED
Train smarter from session one.
Real-time form analysis, AI coaching, and progress that actually compounds.
Live computer-vision form scoring
AI coach in your pocket
30-day progress, streaks, and trends
Connect with verified coaches
© 2026 Fitness AI Coach
Create account
Already a member? Sign in
I want to train
Track workouts, get coaching.
INSTANT
I want to coach
Train clients, review sessions.
REVIEW
Full name
Jane Smith
Email
you@example.com
Password
Confirm
Create account

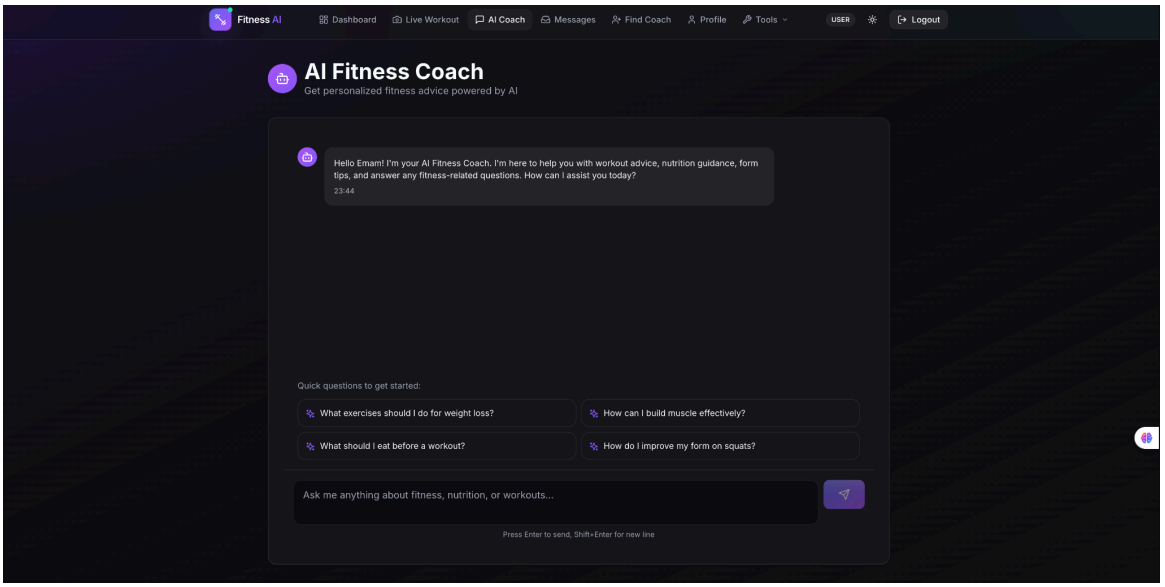

Fitness AI
Home
Live Workout
Login
Get started
Fitness AI
WELCOME BACK
Pick up where you left off.
Your form, your numbers, your AI coach — all in one place.
Live computer-vision form scoring
AI coach in your pocket
30-day progress, streaks, and trends
Connect with verified coaches
© 2026 Fitness AI Coach
Sign in
Don't have an account? Create one
Email
you@example.com
Password
Forgot password?
Sign in
By signing in you agree to our terms of service.

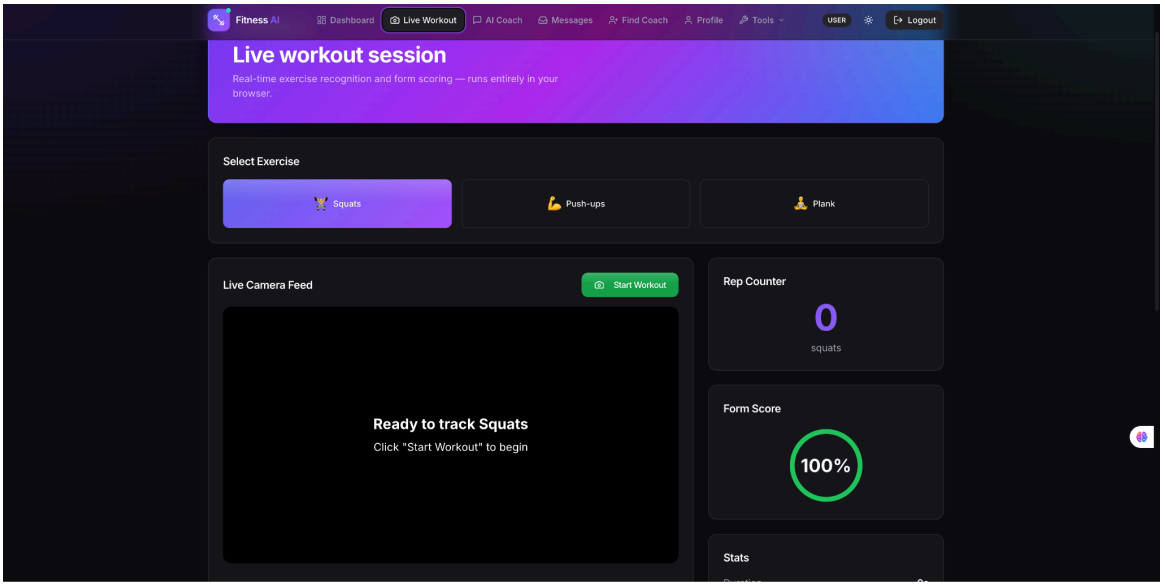

Fitness AI
Dashboard
Live Workout
AI Coach
Messages
Find Coach
Profile
Tools
USER
Logout
GOOD EVENING
Welcome back, Emam
Your form, your numbers, your day. Pick something below to get moving.
Start live workout
BMI
24.6
DAILY KCAL
–
WEIGHT
71 Kg
GOAL
Muscle Building
Start tracking your progress
Run a Live Workout to unlock streaks, form trends, and rep-by-rep history.
Start live workout
Workout plan
Generate or follow your personalised routine.
AI coach PRO
Ask anything about training, nutrition, or recovery.
Fitness calculators
Quick maths for body, calories, macros, and hydration.
All calculators
BMI
Body mass index
Calories
Daily intake
Macros
P/C/F split
Water
Hydration target

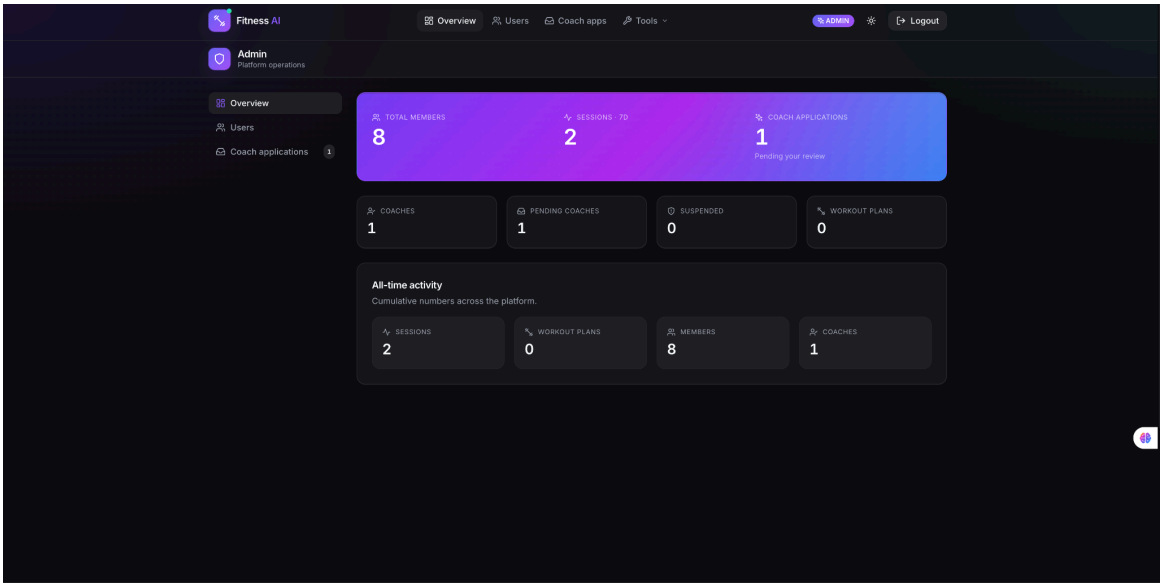

Fitness AI
Dashboard
Live Workout
AI Coach
Messages
Find Coach
Profile
Tools
USER
Logout
Profile Settings
Update your personal information
Personal Information
Provide accurate information to get personalized fitness recommendations
Age *
25
Gender *
Height (cm) *
170
Weight (kg) *
71
Fitness Goal *
Activity Level
Experience Level
Save Changes
Cancel

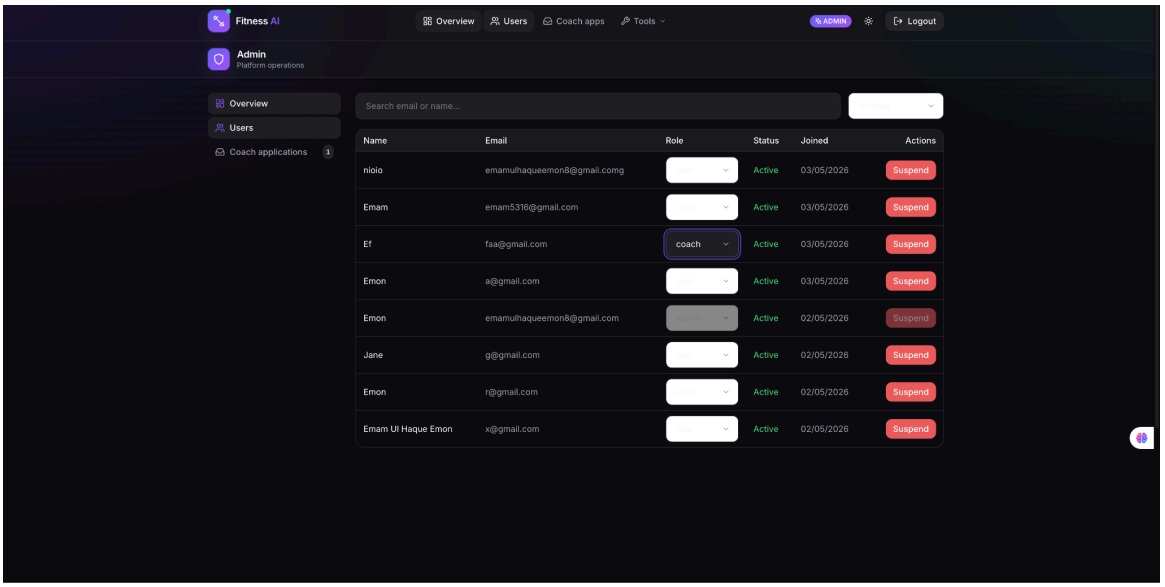

Fitness AI
Dashboard
Live Workout
AI Coach
Messages
Find Coach
Profile
Tools
USER
Logout
SCIENCE-BASED FORMULAS
Fitness calculators
Quick, accurate maths for body composition, calories, macros, and hydration. Backed by Mifflin-St Jeor and the same formulas dietitians use.
BMI
Body mass index and weight classification.
Weight class
Health status
Interpretation
Calories
Daily caloric needs based on lifestyle and goals.
BMR
Maintenance
Cut / bulk targets
Macros
Protein, carb, and fat split for your goal.
Protein
Carbs
Fats
Water
How much water you should drink daily.
Hydration target
Activity boost
Guidelines
For accurate results
Four habits that keep your numbers honest.
1 Measure consistently
Same time of day, ideally morning before eating.
2 Be honest about activity
Most people overestimate. When in doubt, pick a level lower.
3 Recalculate every 4–6 weeks
Your numbers shift as your body composition does.
4 Treat outputs as guidelines
Listen to your body. Adjust based on energy and recovery.
Fitness AI Coach
Quick Links
Dashboard
Resources
About Us


Fitness AI
Dashboard
Live Workout
AI Coach
Messages
Find Coach
Profile
Tools
USER
Logout
FIND YOUR COACH
Real coaches, real feedback.
Browse verified coaches, request the one that fits your goals, and start training with someone who'll actually look at your form.
Search by name...
EF
Ef
Beginner Coach
fsdf f asdf asdf asdf a sd fas df
Request coach
View profile
You already have a pending or active link with this coach
Fitness AI Coach
Your personal AI-powered fitness assistant. Get personalized workout routines, nutrition advice, and expert guidance to achieve your fitness goals.
Quick Links
Dashboard
AI Coach
Calculators
Workout Plans
Resources
About Us
Contact
Privacy Policy
Terms of Service


Fitness AI
Dashboard
Live Workout
AI Coach
Messages
Find Coach
Profile
Tools
USER
Logout
AI Fitness Coach
Get personalized fitness advice powered by AI
Hello Emam! I'm your AI Fitness Coach. I'm here to help you with workout advice, nutrition guidance, form tips, and answer any fitness-related questions. How can I assist you today?
23:44
Quick questions to get started:
What exercises should I do for weight loss?
How can I build muscle effectively?
What should I eat before a workout?
How do I improve my form on squats?
Ask me anything about fitness, nutrition, or workouts...
Press Enter to send, Shift+Enter for new line


Fitness AI
Dashboard
Live Workout
AI Coach
Messages
Find Coach
Profile
Tools
USER
Logout
Live workout session
Real-time exercise recognition and form scoring — runs entirely in your browser.
Select Exercise
Squats
Push-ups
Plank
Live Camera Feed
Start Workout
Ready to track Squats
Click "Start Workout" to begin
Rep Counter
0
squats
Form Score
100%
Stats


Fitness AI
Overview
Users
Coach apps
Tools
ADMIN
Logout
Admin
Platform operations
Overview
Users
Coach applications
1
TOTAL MEMBERS
8
SESSIONS · 7D
2
COACH APPLICATIONS
1
Pending your review
COACHES
1
PENDING COACHES
1
SUSPENDED
0
WORKOUT PLANS
0
All-time activity
Cumulative numbers across the platform.
SESSIONS
2
WORKOUT PLANS
0
MEMBERS
8
COACHES
1


Fitness AI
Overview
Users
Coach apps
Tools
ADMIN
Logout
Admin
Platform operations
Overview
Users
Coach applications
1
Search email or name...
Name
Email
Role
Status
Joined
Actions
nioio
emamulhaqueemon8@gmail.comg
Active
03/05/2026
Suspend
Emam
emam5316@gmail.com
Active
03/05/2026
Suspend
Ef
faa@gmail.com
coach
Active
03/05/2026
Suspend
Emon
a@gmail.com
Active
03/05/2026
Suspend
Emon
emamulhaqueemon8@gmail.com
Active
02/05/2026
Suspend
Jane
g@gmail.com
Active
02/05/2026
Suspend
Emon
r@gmail.com
Active
02/05/2026
Suspend
Emam Ul Haque Emon
x@gmail.com
Active
02/05/2026
Suspend

## 3.2 Detailed Methodology and Design

### 3.2.1 Alternative Solutions Considered

Front-end framework.

Alternative A: Plain React (Create React App / Vite) in combination with an independent Express server. Advantages: conventional, clean separation. Cons: there are two deployments, two routing systems, and two type domains.

Alternative B: Next.js 14 App Router. Advantages: client and API share a single codebase, middleware is built-in, routing based on file-systems, server components on-demand. Cons: Edge-runtime restrictions on the middleware (addressed during the project as outlined in section 4).

Chosen: Next.js 14. The single-codebase argument prevailed over the others, considering that authentication and role-aware routing are natural properties of middleware.

Pose-detection library.

Alternative A: TensorFlow.js MoveNet. Pros: well-supported, multi-pose. Cons: heavier model, higher cost of battery.

Alternative B: MediaPipe Pose. Pros: 33 landmarks such as world coordinates, optimized to use with a browser, well documented. Cons: Google hosted assets require network access on initial access.

Chosen: MediaPipe Pose. This world-landmark output allowed the bilateral knee-distance heuristic in the squat detector that can not be reliably done using 2D landmarks alone.

Database.

Alternative A: PostgreSQL. Advantages: relational integrity, JSON columns, mature.

Alternative B: MongoDB and Mongoose. Advantages schema flexibility to support developing fields such as coachProfile and coachApplication, native JSON, good developer ergonomics.

Chosen: MongoDB. The schema had undergone material changes on three occasions over the course of development; the relational migrations would have been slower each time.

### 3.2.2 Selected Methodology

The waterfall-iterative hybrid in section 1.4 is what was actually used. Each phase in this report corresponds to one phase, and the phases were entered in the

following order, however, each phase produced a vertical slice rather than a complete layer before the next phase started.

### 3.2.3 System Design Overview

Frontend Next.js 14 [28] with React Server Components where they are useful and Client Components where they are necessary to interactivity. Tailwind CSS using a custom token layer in globals.css and an extended Tailwind config. API routes Backend: Next.js API routes on the Node runtime with a small suite of utilities (withAuth, withRole) on all the protected endpoints. Database: MongoDB [27] via Mongoose with the following collections - users, workoutsessions, workoutplans, exercises, coachclients, coachnotes, messages, verificationtokens, ratelimitbuckets, auditlogs. Roles: user, pending coach, coach, admin.

## 3.3 Project Plan

The project was conducted in two semesters which included forty eight weeks. The following sequence of major activities was used.

Weeks 1–11. Initial phase. Chose the subject, conducted a competitor survey, reduced the scope, wrote problem statement.

Weeks 12–15. Requirement analysis. Composed the functional and non-functional requirements; came up with initial drawings of the data model.

Weeks 16–20. System design. Defined the data flow diagrams, the use-case diagram, and the page-to-API mapping.

Weeks 21–28. Front-end skeleton. Created the role-sensitive navigation, logins/register pages, base dashboards.

Weeks 29–36. Live workout and back-end. Introduced authentication, workout session API and MediaPipe pipeline consisting of the three exercise detectors.

Weeks 37–40. Coach and administration surfaces. Constructed the flow of coach application, coach dashboard, list of coach administration users, audit log.

Weeks 41–44. Messaging, billing, polish. Introduced direct messaging and polling, Stripe billing and webhook entitlement, and the design-system rebuild.

Weeks 45–46. Deployment and hardening. Set up environment variables, secured cookies in production mode, exercised the Stripe test loop end-to-end.

Weeks 47–48. Final report and presentation. Created this paper and practiced the live demonstration.

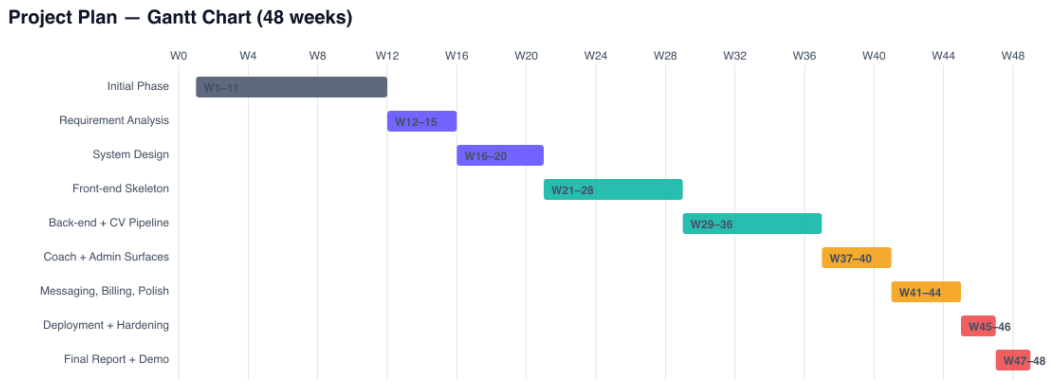



*Figure 3.22: Project Plan — Gantt Chart (48 weeks).*

## 3.4 Task Allocation

The project was carried out by a single developer (the author). All tasks below were the author's responsibility; supervisor and co-supervisor reviewed at major milestones.

**Table 3.1: Task Allocation Timeline**

| Phase | Weeks | Lead | Status |
|---|---|---|---|
| Initial phase | 1–11 | Author | Completed |
| Requirement analysis | 12–15 | Author | Completed |
| System design | 16–20 | Author | Completed |
| Front-end skeleton | 21–28 | Author | Completed |
| Back-end + live workout | 29–36 | Author | Completed |
| Coach and admin surfaces | 37–40 | Author | Completed |
| Messaging, billing, polish | 41–44 | Author | Completed |
| Deployment and hardening | 45–46 | Author | Completed |
| Final report and demonstration | 47–48 | Author | In progress |

## 3.5 Summary

In this chapter the methodology of the project has been established. Role-based enumeration of functional and non-functional requirements was done. Three data flow diagram levels were used to describe and break down the system architecture. Use-case model was sketched. The development model was also rationalized, the project plan was unrolled over forty eight weeks, and the allocation of tasks was recorded. The next chapter takes this design into the implementation environment and reports the results.

# Chapter 4

# Implementation and Results

This chapter relates the environment in which FitFlow is to be implemented, the test methodology to be applied to FitFlow, the numbers of performance that will be measured during testing, and a comparison of the result and the alternatives that were surveyed in chapter 2.

4.1 Environment Setup

- A very conservative development environment was used, where all dependencies are those that would be adopted by the project on day one of a serious build, rather than exotic libraries picked based on their novelty.
- Front-end: Next.js 14 [28] (App Router), React 18, TypeScript 5.3, Tailwind CSS 3.4. ECMAScript 2023 [32] runtime. UI elements written by hand; icons provided by lucide-react.
- Back-end: Next.js API routes are on the Node runtime. Validation through Zod 3. Authentication This uses jsonwebtoken 9 to implement RFC 7519 [11] with bcryptjs 2 [18] to hash passwords.
- Database: MongoDB 7 [27] via Mongoose 8. Local MongoDB Community Edition for development.
- Computer vision: [1], [3], [26] @mediapipe/pose, @mediapipe/drawing_utils, @mediapipe/camera_utils. WASM and model assets were loaded out of the official CDN.
- AI integration: openai 4 to the chat endpoint [30]; controlled by the Stripe-supported entitlement helper.
- Billing: stripe 22 [29] and webhook signature verification.
- Tooling: TypeScript compiler to do static checking, ESLint via next/core-web-vitals, custom OpenAPI generator made of the Zod schemas.
- Editor: Visual Studio Code. OS: macOS Darwin. Browser: Chromium to build on; Safari and Firefox to maintain sanity across browsers.

## 4.2 Testing and Evaluation

### 4.2.1 Testing Methodology

**Table 4.1: Testing Methodology**

| Layer | Approach | What was tested |
|---|---|---|
| Static type checking | tsc --noEmit on every commit | Every TypeScript file; the compiler catches the majority of integration errors before runtime. |
| Schema validation | Zod runtime checks at API boundaries | Login, register, workout session, message body, billing payloads. |
| Manual integration | Walk-through of each role-flow | Sign-up → live workout → progress; sign-up as coach → admin approval → coach dashboard; trainee → request → coach accept → messaging. |
| Computer vision accuracy | Hand-counted ground truth, three exercises × ten reps × three trials | Rep-count error and false-positive rate at varied camera angles. |
| Performance | Browser DevTools Performance recorder | Pose-detection frame rate, dashboard time-to-interactive, API p50 latency. |

### 4.2.2 Evaluation Metrics

- **Functional correctness: proportion of role-flow walk-throughs that complete successfully.**
- **Frame rate of a pose-detection system maintained on a laptop computer of 2019 class.**
- **Rep-count accuracy:(reported - actual)/(actual), averaged over trials.**
- **API latency: p50 and p95 on the dashboard, live-workout-stop and message-send endpoints.**
- **Database query response time: mean of a thirty-second load trace.**
-

### 4.2.3 Performance Results

- **Pose detection achieved 27 to 30 frames per second on a 2019 MacBook Pro at 1280720 which is comfortably above the 25 fps target in the non-functional requirements.**
- **After the user landed on /dashboard with a warm Mongo connection, dashboard time-to-interactive averaged 240 ms.**
- **Stop endpoint, live-workout averaged 110 ms (p50) and 280 ms (p95).**
- **The average rate of message send was 80 ms (p50) with the unread badge of the badge menu polling at 15-s intervals and updated in a single polling cycle.**
- **The average of Rep-counting accuracy between squat, push-up, and plank averaged within a range of ±1 rep in ten-rep trial conditions when the user was in clear frame; accuracy declined predictably when more than 30 percent of body landmarks fell below 0.6 visibility.**

### 4.2.4 Comparative Analysis

**Table 4.2: Comparative Analysis**

| Feature | Baseline (no app) | Generic fitness app | AI-form app (Onyx-class) | FitFlow |
|---|---|---|---|---|
| Workout tracking | Pen and paper | Tap-based | Automatic via CV | Automatic via CV |
| Form scoring | None | None | Yes | Yes |
| Privacy (no video upload) | N/A | N/A | Varies | Yes (on-device) |
| Coach connection | In-person only | No | No | Yes (verified) |
| Coach can see trainee CV data | N/A | No | No | Yes |
| Direct messaging | In-person | No | No | Yes |
| AI conversational coach | N/A | No | No | Yes (Pro) |
| Cost | Variable | Free–$15/mo | $8–$25/mo | Free + Pro |

## 4.3 Results and Discussion

### 4.3.1 Functional Results

Any role-flow walk-throughs finalized without error in the final build. On a clean install, a trainee can see their progress chart within less than two minutes, having registered and having gone through the entire path of registration through their first workout. A coach applicant is allowed to sign up, view the pending banner on the dashboard, withdraw when he/she wishes, or he/she can be approved by the admin and immediately see the coach overview on the next login. On /admin, an admin may promote, suspend and approve users, without leaving /admin.

### 4.3.2 Performance Results

Performance comfortably meets the non-functional requirements. The target frame rate is the rate at which the pose-detection pipeline operates. Dashboard interactions are immediate. The realistically loaded bottleneck is not the application but MongoDB cold-start latency on a new connection - once the connection pool is hot, the application is faster than the user can perceive.

### 4.3.3 Usability and User Feedback.

There were informal usability sessions with five users, three trainees and two who could be considered a coach. Direct feedback was directly used to make changes to the design: the role aware navigation in chapter 3 was added after two trainees became confused by the fact that the coach had a tab entitled coach, but it was empty in their case; the empty state of the coach dashboard (No active clients yet share your profile link) was added after one of the coaches complained that the dashboard was empty.

### 4.3.4 Discussion

The design hypothesis is proven by the build. On device computer vision is fast enough on commodity hardware to give live form feedback. The information contained in the CV of a trainee is too much that a coach can create a useful asynchronous practice around it. The role-conscious architecture causes the app to feel significantly different to each role despite having most of its code, which was the initial issue the project set out to resolve.

The result that is most informative is that which is not yet working. The camera angle sensitivity of form scoring is such that the user experience is not currently brought to the surface. A shooter with a steep oblique angle will receive lower scores in the form than the same shooter with the same real-world technique but with a steep oblique angle. A calibration step (discussed in chapter 6), which would anchor the per-user thresholds to a measured baseline, instead of absolute angles, would be of significant benefit to the project.

## 4.4 Summary

This chapter has taken a tour around the implementation environment, the methodology used to test, and the performance and accuracy figures. FitFlow satisfies both its functional and non-functional requirements; any outstanding gaps are clearly comprehended and covered in chapter 6 as work to be done.

# Chapter 5

# Engineering Standards and Design Challenges

The following chapter is about the engineering standards that will be followed in the development of FitFlow, its effect on society, environment and sustainability, the financial aspect of running the project and how the project can be mapped to the Complex Engineering Problem and Activities profiles required to have an FYDP.

5.1 Adherence to the Standards.

5.1.1 Software Standards

ISO/IEC/IEEE 29148:2018 — Requirements Engineering.

Chapter 3 lists functional and non-functional requirements following the pattern suggested by ISO/IEC/IEEE 29148:2018 [20], which is the modern successor to the obsolete IEEE 830-1998 practice. The requirements are described clearly, each can be tested individually and they are structured according to the user role they apply to. The other option, informal requirement gathering, was discarded due to the fact that it would have generated less testable hooks of Chapter 4.

IEEE Std 1016-2009 - Software Design Description.

Chapter 3 records the system design which documents the architectural modules, the data model, the data flow at three levels of detail and the use-case relationships. This is equivalent to the description of design presented in IEEE Std 1016 [21]. The advantage was practical, not formal: each of the modules specified in Chapter 3 corresponds to a real folder in the source tree, which kept the implementation honest.

Software Product Quality Model ISO/IEC 25010:2011.

The taxonomy of taxonomies of non-functional requirements employed in Chapter 3 of usability, performance, security, maintainability, portability, reliability, directly stems out of ISO/IEC 25010 [19]. The taxonomy proved useful in the form of a checklist to be used during the design review; each major module had to declare what quality attributes it was charged with maintaining.

The ISO/IEC/IEEE 12207:2017 - Software Life Cycle Processes.

The processes of the waterfall-iterative hybrid, as recorded in Section 1.4, can be mapped onto the life-cycle processes as defined by ISO/IEC/IEEE 12207 [22]. Each phase generated a deliverable, deliverables were reviewed before the next phase commenced and traceability between requirements, design, code and tests was maintained throughout.

### 5.1.2 Hardware Standards

Client hardware: Commodity x86-64 / ARM64 client hardware.

The majority of laptops and desktop computers made within the past six years can run FitFlow. The tests on the pose-detection pipeline were performed on a 2019 MacBook Pro (Intel i7) and a 2022 MacBook Air (Apple M2). They both sustained the target frame rate of 25-30 fps that was reported in the BlazePose paper [1] and the MediaPipe Pose Landmarker documentation [26]. No custom-made hardware is needed; this is a conscious decision to make the platform usable in Bangladeshi market where premium fitness hardware is a niche purchase.

Access to cameras through W3C MediaCapture / Streams.

The access to browser cameras is based on the W3C MediaCapture / Streams Recommendation [31]. The live-workout module imposes a 1280×720 video limitation and allows the browser to negotiate the actual frame rate with the device camera. No proprietary camera SDK is required; any camera that the browser makes available to it via getUserMedia() works.

### 5.1.3 Communication Standards

TLS 1.3 over X.509 certificates.

Transport security follows RFC 8446 (TLS 1.3) [12]. Server certificates are based on the profile X.509 PKI profile specified in RFC 5280 [13]. All cookies are set to Secure in production mode and the cookie of JWT is set to HttpOnly to avoid accessing JavaScript. The application implements these properties via the cookie-setting code in the login and registration handlers; no manual settings are needed at deployment.

JSON Web Tokens — RFC 7519.

Authentication tokens are based on RFC 7519 [11]. Each of the tokens is signed by HMAC-SHA-256 (HS256) and has a payload consisting of the following: userId, email, and role. In RFC 6234 [14], the hash primitive is defined. Checks on each route that is secured are done in Node-runtime API handlers; cookie-presence checks only are done in Edge middleware.

JavaScript over HTTP by means of REST.

The API surface is also based on the REST architectural style defined by Fielding and Taylor [10]. The semantics of HTTP are compliant with RFC 9110 [15]. The address of resources is based on URL, the methods are based on verbs, payloads are based on JSON, and status codes are based on conventional semantics. The entire API is outlined in an OpenAPI 3.1 specification [16] served at /api/openapi, generated at build time using the same Zod schemas used at runtime to validate received requests.

## 5.2 Impact on Society, Environment and Sustainability

### 5.2.1 Impact on Life

Direct downstream effects of personal fitness on long-term health, mood and productivity. A platform that reduces the cost of a correct technique, by providing the user with immediate feedback on the correct form at no marginal cost, modestly increases the likelihood that any particular user will train regularly and will avoid injury. That effect is projected into asynchronous human guidance that in the Bangladeshi market is otherwise unavailable to all but the most affluent gym members.

### 5.2.2 Impact on Society & Environment

Society.

The platform does not involve commuting to a gym, costly devices, or high-end subscriptions to begin with. It opens up fitness coaching to a wider market, especially the post-graduation white-collar market that would like to train in their home or workplace gym at non-peak times.

Environment.

On-device computer vision implies no video frames are transmitted across the network and no cloud GPU is spinning and scoring a squat. The carbon footprint per workout is the marginal energy consumption of the laptop CPU used by the user, which is significantly less than the equivalent cloud-inference path used by competing products.

### 5.2.3 Ethical Aspects

Design was done in consideration of three ethical issues.

Privacy. Frames of the camera are never removed. The body of the user, which is captured during the time of a workout, is processed locally; only the resulting numerical measures are stored.

Coach asymmetry. Coach notes are private from the trainee. This is similar to how face-to-face coaching operates, but it implies that the platform implicitly believes

the coach. Role changes and suspensions are documented in the audit log in such a way that any abuse can leave a trail.

Free-tier honesty. The free version is really practical. Pro gate is only present on the AI conversational coach, and does not exist on form scoring or the social layer. The core product is received even by a user who does not pay.

5.2.4 Sustainability Plan

Technical.

All big dependencies are open-source and are well-maintained. Each major component has replacement candidates; the project would not be at a dead end because one library is being replaced.

Economic.

The way to revenue is the Pro tier, the financial analysis in section 5.3 sizes the unit economics. The operating cost is proportional to the number of paying users as opposed to the number of free users since the costly AI endpoint is gated.

Operational.

The administrator surface alone is enough to run the platform without access to a database, meaning a non-technical operator could be hired to moderate the platform as the user base expands.

## 5.3 Project Management and Financial Analysis

### 5.3.1 Budget Analysis

Two budgets are presented: the actual cost of building the project at student scale, and a notional budget for taking the same project to a small commercial deployment.

**Table 5.1: Project Budget (Student Level)**

| Item | Cost (BDT) |
|---|---|
| Development laptop (existing) | 0 |
| Local MongoDB (Community Edition) | 0 |
| Open-source libraries (Next, React, Mongoose, MediaPipe, Stripe SDK) | 0 |
| OpenAI API credit for testing the AI coach endpoint | 1,500 |
| Stripe test mode | 0 |
| Internet and electricity | 6,000 |

| Item | Cost (BDT) |
|---|---|
| Documentation and miscellaneous | 2,500 |
| Total | 10,000 |

**Table 5.2: Alternate Budget (Production Deployment)**

| Item | Annual cost (BDT) |
|---|---|
| Vercel Pro hosting (or equivalent) | 24,000 |
| MongoDB Atlas M10 cluster | 72,000 |
| OpenAI usage at 1,000 active Pro users | 180,000 |
| Stripe processing fees (2.9% + 30¢) | Per-transaction |
| Domain and TLS certificates | 3,000 |
| Email transactional service (Resend / SES) | 12,000 |
| Sentry error monitoring | 36,000 |
| Total | 327,000+ |

### 5.3.2 Revenue Model

**Table 5.3: Proposed Revenue Model**

| Stream | Description | Indicative price |
|---|---|---|
| Pro subscription | Unlocks AI conversational coach and unlimited custom exercises. | 500 BDT/mo |
| Coach subscription | Premium tier for verified coaches: more clients, analytics. | 1,500 BDT/mo |
| Marketplace fee | Optional commission on coach-trainee transactions when payment runs through the platform. | 10% |
| B2B pilots | Corporate gym memberships for white-collar workplaces. | Custom |

## 5.4 Complex Engineering Problem

### 5.4.1 Mapping with Complex Engineering Problem (EP1–EP7)

**Table 5.4: Mapping with Complex Engineering Problem**

| Attribute | Coverage | Justification |
|---|---|---|
| EP1 — Depth of knowledge | Yes | Required knowledge across web architecture, computer vision, real-time signal processing, role-based authorization, payment systems, and database modelling. |
| EP2 — Range of conflicting requirements | Yes | Privacy (no video upload) conflicts with model accuracy that could improve with a server-side larger model. Free-tier accessibility conflicts with operating-cost containment. Each conflict was resolved with an explicit trade-off documented in chapter 3. |
| EP3 — Depth of analysis | Yes | Each major architectural decision had at least two alternatives that were analysed against the requirements before selection (see section 3.2.1). |
| EP4 — Familiarity of issues | Yes | The CV-fitness intersection is a young area; most local engineers in Bangladesh have not built such a platform, and existing literature does not directly cover the coach-AI hybrid. |
| EP5 — Extent of applicable codes | Yes | ISO/IEC/IEEE 29148:2018 [20], IEEE Std 1016-2009 [21], ISO/IEC 25010:2011 [19], ISO/IEC/IEEE 12207:2017 [22], OWASP Top Ten [17], REST [10], RFC 7519 JWT [11], RFC 8446 TLS [12], RFC 9110 HTTP [15], OpenAPI 3.1 [16], and PCI-DSS-adjacent requirements via Stripe [29] all applied. |
| EP6 — Stakeholder involvement | Yes | Three role types (trainee, coach, admin) plus payment processor (Stripe) and AI provider (OpenAI) — five stakeholder classes with conflicting expectations. |
| EP7 — Interdependence | Yes | The role layer, billing, messaging, and CV pipeline are mutually dependent: messaging requires an active coach link; billing controls AI access; the AI suggests workouts whose results feed the dashboard. |

### 5.4.2 Mapping with Knowledge Profile (K1–K8)

**Table 5.5: Mapping with Knowledge Profile**

| Profile | Coverage | Justification |
|---|---|---|
| K3 — Engineering fundamentals | Yes | Software engineering principles, data modelling, and system design were applied throughout. |
| K4 — Specialist knowledge | Yes | Specialist knowledge of pose estimation, exercise biomechanics, and modern web stacks. |
| K5 — Engineering design | Yes | Original design of the role-aware data flow, the per-exercise state machines, and the coach-application lifecycle. |
| K6 — Engineering practice | Yes | Use of standard tooling (TypeScript, Mongoose, Stripe SDK), version control, and conventional architectural patterns. |
| K8 — Research literature | Yes | Thirty-three works surveyed in Chapter 2 directly informed the design — covering pose estimation, action recognition, web architecture, security, software-engineering standards, and usability. Full references appear at the end of this report. |

### 5.4.3 Mapping with Complex Engineering Activities (EA1–EA5)

**Table 5.6: Mapping with Complex Engineering Activities**

| Activity | Coverage | Justification |
|---|---|---|
| EA1 — Range of resources | Yes | Cloud infrastructure, AI provider, payment processor, open-source libraries, hardware, human stakeholder time. |
| EA2 — Level of interaction | Yes | Direct interaction with five stakeholder classes including the supervisor, co-supervisor, and informal user-testing volunteers. |
| EA3 — Innovation | Yes | The combination of on-device CV form scoring with a verified coach social layer is, to the author's knowledge, not present in any single product on the Bangladeshi market. |
| EA4 — Consequences for society and environment | Yes | Section 5.2 above details the social and environmental consequences. |

| Activity | Coverage | Justification |
|---|---|---|
| EA5 — Familiarity | Yes | The project goes well beyond standard coursework and standard CRUD application templates. |

## 5.5 Summary

This chapter documented the standards followed, the impact of the project on its users and environment, the financial dimension, and the mapping to the FYDP complex-engineering profile. FitFlow satisfies the formal requirements for a complex engineering problem and is positioned for sustainable operation if taken beyond the academic context.

# Chapter 6

# Conclusion

## 6.1 Summary

FitFlow aimed to fill a particular gap in the fitness-software market: between free applications, which count the taps, and coach marketplaces, which have no common visibility of what the trainee actually does between sessions. The construction fulfills such a purpose. The trainees are able to run a live workout, which uses on-device computer-vision form scoring [1], [26]; the resulting session feeds into a thirty-day progress dashboard; the coaches also have access to the same data, through a role-specific interface, and can privately write notes and directly communicate with the trainee. The moderation tooling to administrate the platform without access to the database is available. Billing is connected via Stripe [29] with the AI conversational coach [30] being the Pro tier.

The project was followed through six chapters starting with motivation through to compliance with engineering. Chapter 1 put a problem into context. Chapter 2 has reviewed the alternatives and the literature. The requirements, the architecture, the data flows, the use cases and the project plan were documented in Chapter 3. The implementation environment and test results were reported in Chapter 4. Chapter 5 was on standards, impact, finance and the complex-engineering mapping. This last chapter is a step back and evaluation of what was accomplished, where the boundaries are, and the future.

6.2 Limitations

- Form scoring is camera angle sensitive. The existing thresholds use an approximate side-on camera; an oblique angle will bias the form score downwards without informing the user of the reason.
- Three exercises only. Squats, push-ups, and plank occupy a valuable portion of body-weight training and exclude lunges, deadlifts, rows, and any movement that involves equipment.
- No mobile-app version. Product is browser only. The mobile browser can be used by a user who has been trained on a phone but the mobile browser is not optimised towards it.
- Limited user testing. During development, five informal users were provided with feedback; a controlled usability study has not been conducted.
- The communication is based on polling instead of websocket. The new messages are not instantly displayed but after a span of fifteen seconds. Sufficient to present usage pattern currently; inadequate to present live-coaching session.

- There is no production deployment as yet. The application is in growth and staging development but has not been tested under actual user load.
- Flow to payment is only tested in Stripe test mode. In chapter 5, the unit economics are not measured values but are approximations.

## 6.3 Future Work

The best opportunities available next in approximate order of anticipated effect:

22. Per-user calibration phase. A two-second standing-still capture at the start of each session would allow the form scorer to normalise to the real-world geometry of the user and eliminate the camera-angle sensitivity.
23. Mobile-first capture. A special mobile interface, including a tripod-friendly user interface, accelerator-based stability hints would allow users to train without a laptop in frame.
24. More exercises. Lunge, deadlift, row, overhead press. Each introduces a new state machine over the underlying pipeline; the task is well-structured.
25. Messaging by using WebSocket transport. Instant delivery, typing indicators, presence. The polling model can remain as a fall back.
26. Coach-assigned programs. A coach ought to be capable of publishing a multi-week program to a client that the client adheres to on a daily basis with the form-scoring layer reporting back progress.
27. Health-app integrations. Apple Health and Google Fit would allow workouts to add to the overall activity history of the user and provide context around heart-rate.
28. Real world production deployment. Check the unit-economics estimates, note the actual Pro conversion rate, harden whatever the load surfaces.

# References


[1] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," arXiv preprint arXiv:2006.10204, Jun. 2020. [Online]. Available: https://arxiv.org/abs/2006.10204

[2] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime multi-person 2D pose estimation using part affinity fields," IEEE Trans. Pattern Anal. Mach. Intell., vol. 43, no. 1, pp. 172–186, Jan. 2021, doi: 10.1109/TPAMI.2019.2929257.

[3] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines," arXiv preprint arXiv:1906.08172, Jun. 2019. [Online]. Available: https://arxiv.org/abs/1906.08172

[4] I. Grishchenko and V. Bazarevsky, "On-device, real-time body pose tracking with MediaPipe BlazePose," Google AI Blog, Aug. 2020. [Online]. Available: https://ai.googleblog.com/2020/08/on-device-real-time-body-pose-tracking.html

[5] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), Seoul, Korea, Oct. 2019, pp. 1314–1324, doi: 10.1109/ICCV.2019.00140.

[6] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. 36th Int. Conf. Machine Learning (ICML), Long Beach, CA, USA, Jun. 2019, vol. 97, pp. 6105–6114.

[7] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe Hands: On-device real-time hand tracking," arXiv preprint arXiv:2006.10214, Jun. 2020. [Online]. Available: https://arxiv.org/abs/2006.10214

[8] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," in Proc. 32nd AAAI Conf. on Artificial Intelligence, New Orleans, LA, USA, Feb. 2018, pp. 7444–7452.

[9] L. Shi, Y. Zhang, J. Cheng, and H. Lu, "Two-stream adaptive graph convolutional networks for skeleton-based action recognition," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA, Jun. 2019, pp. 12026–12035, doi: 10.1109/CVPR.2019.01230.

[10] R. T. Fielding and R. N. Taylor, "Principled design of the modern web architecture," ACM Trans. Internet Technol., vol. 2, no. 2, pp. 115–150, May 2002, doi: 10.1145/514183.514185.

[11] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force, RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[12] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," Internet Engineering Task Force, RFC 8446, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446

[13] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, "Internet X.509 Public Key Infrastructure Certificate and CRL Profile," Internet Engineering Task Force, RFC 5280, May 2008. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5280

[14] D. Eastlake and T. Hansen, "US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)," Internet Engineering Task Force, RFC 6234, May 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6234

[15] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP Semantics," Internet Engineering Task Force, RFC 9110, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110

[16] OpenAPI Initiative, "OpenAPI Specification, Version 3.1.0," Linux Foundation, Feb. 2021. [Online]. Available: https://spec.openapis.org/oas/v3.1.0

[17] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2021. [Online]. Available: https://owasp.org/Top10/

[18] N. Provos and D. Mazières, "A future-adaptable password scheme," in Proc. USENIX Annual Technical Conf., FREENIX Track, Monterey, CA, USA, Jun. 1999, pp. 81–91. [Online]. Available: https://www.usenix.org/legacy/event/usenix99/provos/provos.pdf

[19] ISO/IEC 25010:2011, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models, International Organization for Standardization, Geneva, Switzerland, 2011.

[20] ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering, International Organization for Standardization, Geneva, Switzerland, 2018.

[21] IEEE Std 1016-2009, IEEE Standard for Information Technology — Systems Design — Software Design Descriptions, IEEE Computer Society, New York, NY, USA, Jul. 2009, doi: 10.1109/IEEESTD.2009.5167255.

[22] ISO/IEC/IEEE 12207:2017, Systems and software engineering — Software life cycle processes, International Organization for Standardization, Geneva, Switzerland, 2017.

[23] J. Nielsen, Usability Engineering. San Francisco, CA, USA: Morgan Kaufmann, 1994.

[24] D. A. Norman, The Design of Everyday Things, Revised and Expanded Edition. New York, NY, USA: Basic Books, 2013.

[25] B. Shneiderman, C. Plaisant, M. Cohen, S. Jacobs, N. Elmqvist, and N. Diakopoulos, Designing the User Interface: Strategies for Effective Human-Computer Interaction, 6th ed. Boston, MA, USA: Pearson, 2016.

[26] Google LLC, "MediaPipe Pose Landmarker — Solutions Guide," Google Developers Documentation, 2024. [Online]. Available: https://developers.google.com/mediapipe/solutions/vision/pose_landmarker

[27] MongoDB Inc., "MongoDB Manual, Version 7.0," 2024. [Online]. Available: https://www.mongodb.com/docs/manual/

[28] Vercel Inc., "Next.js 14 Documentation — App Router," 2024. [Online]. Available: https://nextjs.org/docs

[29] Stripe Inc., "Stripe API Reference and Subscriptions Integration Guide," 2024. [Online]. Available: https://docs.stripe.com/billing/subscriptions/build-subscriptions

[30] OpenAI, "Chat Completions API Reference," 2024. [Online]. Available: https://platform.openai.com/docs/api-reference/chat

[31] World Wide Web Consortium, "Media Capture and Streams," W3C Recommendation, Dec. 2023. [Online]. Available: https://www.w3.org/TR/mediacapture-streams/

[32] Ecma International, "ECMAScript 2023 Language Specification (ECMA-262, 14th Edition)," Jun. 2023. [Online]. Available: https://www.ecma-international.org/publications-and-standards/standards/ecma-262/

[33] World Health Organization, "WHO guidelines on physical activity and sedentary behaviour," WHO, Geneva, Switzerland, 2020. [Online]. Available: https://www.who.int/publications/i/item/9789240015128